

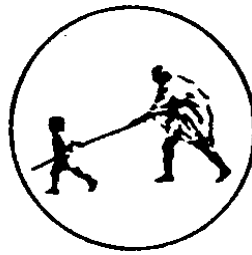
An Internship Report on
Coffee Ordering and Management Platform
&
Infosys Springboard Virtual Internship 6.0

BY

KANCHAN RAJU DHUTADE

Under the Guidance
of
Ms. K. P. Pople

(Department Of Computer Science And Engineering)



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Mahatma Gandhi Mission's College of Engineering, Nanded (M.S.)

Academic Year 2025-26

An Internship Report on
“Coffee Ordering and Management Platform”
&
“Infosys Springboard Virtual Internship 6.0”

Submitted to
DR. BABASAHEB AMBEDKAR TECHNOLOGICAL
UNIVERSITY, LONERE

for fulfillment of the requirement for the degree of

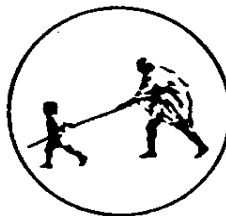
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING

By
KANCHAN RAJU DHUTADE

Under the Guidance
of

Ms. K. P. Pople

(DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING)



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
MAHATMA GANDHI MISSION'S COLLEGE OF ENGINEERING NANDED.

Academic Year 2025-26

CERTIFICATE



This is to certify that the Internship entitled

“Coffee Ordering and Management Platform”

&

“Infosys Springboard Virtual Internship 6.0”

being submitted by MISS. KANCHAN RAJU DHUTADE to the Dr. Babasaheb Ambedkar Technological University, Lonere, for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by her under my supervision and guidance. The matter contained in this report has not been submitted to any other university or institute for the award of any degree.

Ms. K. P. Pople
Project Guide

Dr. A. M. Rajurkar
H.O.D
Computer Science & Engineering

Dr. G. S. Lathkar
Director
MGM's College of Engg. Nanded

INTERNSHIP OFFER LETTER



Dear Learner,

With reference to the **Infosys Springboard Internship 6.0**, you should have now received your project title and meeting details. Please attend all scheduled meetings as per the communicated plan. Ensure that you attend every session and complete your assigned tasks on time. **If you have not received any communication, kindly reach out to your mentor.**

Do not miss this valuable opportunity to complete your virtual internship.

If you need to take leave for academic exams or placement activities, please obtain prior written approval via email.

Project title: Web-Based Coffee Ordering and Management Platform

Mentor email ID: springboardmentor290@gmail.com

Got a query? Write to springboard-support@infosys.com

Regards,
Team Infosys Springboard

Scan the QR code to download
the **Infosys Springboard App**



App Store



Play Store

Follow us on Facebook | Instagram | LinkedIn | X (Twitter)

INTERNSHIP CERTIFICATE



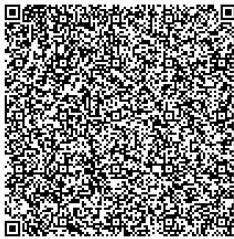
CERTIFICATE OF COMPLETION

presented to

Kanchan Dhutade

for actively participating and completing the mandatory assignment related to
Internship 6.0 (B 13): Web-Based Coffee Ordering and Management Platform

conducted from February 5, 2026 to April 3, 2026



Issued on: Monday, June 1, 2026
To verify, scan the QR code at <https://verify.onwingspan.com>

Satheesha B.N.
Satheesha B. Nanjappa
Senior Vice President and Head
Education, Training and Assessment
Infosys Limited

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my project guide, **Ms. K. P. Pople**, for her invaluable support, guidance, and encouragement throughout this internship. Her profound knowledge and expertise have been instrumental in the successful completion of this work. Her patience and willingness to assist me at every step have greatly enriched my learning experience. Her constructive feedback and insightful suggestions have not only helped me overcome challenges but also motivated me to strive for excellence.

I gladly take this opportunity to thank **Dr. A. M. Rajurkar** (Head of Computer Science & Engineering, MGM's College of Engineering, Nanded). I am heartily thankful to **Dr. G. S. Lathkar** (Director, MGM's College of Engineering, Nanded) for providing facilities during the progress of the project and also for her kind help, guidance and inspiration. Last but not least, I am also thankful to all those who helped, directly or indirectly, in completing this internship project and making it successful.

With Deep Reverence,

KANCHAN RAJU DHUTADE

[B.Tech CSE-B]

ABSTRACT

This report presents the experience and knowledge gained during the internship as a Full-Stack Developer while working on the Coffee Ordering and Management Platform. The application was developed to manage café operations such as menu browsing, cart handling, table reservations, event booking, order management, online payments, and role-based dashboards for customers, café owners, chefs, waiters, and administrators. The internship provided practical exposure to developing a scalable, efficient, and user-friendly web application in a real-world environment.

During the internship, work was carried out on both frontend and backend development using technologies such as React.js, Spring Boot, MySQL, Java, JavaScript, REST APIs, CSS, and GitHub. Frontend contributions included improving responsive layouts, designing interactive user interfaces, managing customer workflows, and implementing features such as cart operations, booking forms, order tracking, and dashboard modules. On the backend side, APIs, controller methods, service-layer logic, validation mechanisms, and database operations were developed to ensure smooth communication between system components.

The internship also provided valuable experience in debugging, testing, API integration, version control, and collaborative development practices. It enhanced technical knowledge in full-stack development while improving problem-solving skills, coding standards, and understanding of maintainable software architecture. Overall, this experience helped bridge the gap between academic learning and industry-level software development practices, preparing for future professional opportunities in the field of software engineering.

TABLE OF CONTENTS

TOPICS	PAGE NO.
INTERNSHIP OFFER LETTER	I
INTERNSHIP CERTIFICATE	II
ACKNOWLEDGEMENT	III
ABSTRACT	IV
TABLE OF CONTENTS	V
LIST OF FIGURES	VIII
Chapter 1. INTRODUCTION	1
1.1 Internship Overview	2
1.2 Company Profile	3
1.3 Training and Skill Development	5
1.4 Projects and Work Experience	6
1.5 Learning Outcomes	7
1.6 Organization of the Report	8
Chapter 2. COMPANY BACKGROUND	9
2.1 Services Provided	10
2.2 Vision, Mission & Certifications	12
2.3 Community Engagement & SB Meetups	14
Chapter 3. TECHNOLOGY STACK	16
3.1 React.js	17
3.1.1 Key Features of React.js	17
3.1.2 Use Case in My Internship	18
3.2 Spring Boot	19
3.2.1 Key Features of Spring Boot	19
3.2.2 Application Workflow in My Internship	20
3.3 MySQL	21
3.3.1 Key Features of MySQL	22
3.3.2 Application Workflow in My Internship	22
Chapter 4. TECHNICAL TRAINING AND ASSIGNMENT	24
4.1 React Fundamentals and Advanced Concepts	26
4.1.1 Core React Concept	26
4.1.2 Asynchronous Operations & API Handling in React	27

4.1.3 React Features and Advanced Frontend Practices	28
4.2 Backend Development with Spring Boot	29
4.2.1 Setting Up the Development Environment, Exploring Spring Boot	29
4.2.2 RESTful API Development with Spring Boot	30
4.2.3 Integrating MySQL and Spring Data JPA for Data Persistence	30
4.2.4 Implementing JWT Authentication and Authorization	31
4.3 Database Operations with MySQL	32
4.3.1 Database Design and Modeling with MySQL	32
4.3.2 Performing CRUD Operations	34
4.3.3 Real-World Use in Internship Projects	34
4.4 Frontend Development with React.js	35
4.4.1 Setting Up and Structuring a React.js Project	36
4.4.2 Building Pages and Components with React	36
4.4.3 Data Fetching and Integration with Backend	37
4.4.4 State Management and Dynamic UI Behavior	37
4.4.5 Real-World Application and Final Project Implementation	37
4.5 Version Control with Git and GitHub	39
4.5.1 Understanding Git Basis and Local Version Control	39
4.5.2 Collaborative Development Using GitHub	40
4.5.3 Best Practices and Workflow Improvements	41
4.5.4 Learning from Real-World Scenarios	41
Chapter 5. INTERNSHIP PROJECT CONTRIBUTION	43
5.1 Initial Product Understanding and System Analysis	44
5.2 Frontend Console Cleanup and API Error Resolution	45
5.3 Backend Code Optimization Through Evaluation	47
5.4 Middleware Implementation for Role-Based Access Control	49
5.5 Implementation of Role-Specific Functionality and Data Flow	50
5.6 Real-Time Chat Functionality Using Socket IO	52
5.7 Report Functionality with Role-Based View Access	54
5.8 UI Design and Responsive Frontend Development	55
5.9 Code Optimization and Version Control Across Branches	57

5.10 Real-Time Bug Fixes and Feature Enhancements	58
CONCLUSION	59
REFERENCES	60

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
1.1	Company Logo	4
1.2	Benefits of Café Management Platform	6
2.1	Services Provided	11
2.2	Company Clients	13
3.1	React.js	17
3.2	Spring Boot	19
3.3	MySQL	21
4.1	Training Program Flow	25
4.2	JSON Web Token	31
5.1	Application Screenshot Look Alike	44
5.2	Coffee Ordering Services	45
5.3	Errors and Warning Resolution	46
5.4	Backend Code Optimization	48
5.5	Middleware Implementation	49
5.6	RBAC Implementation	51
5.7	Real-time Order Tracking	53
5.8	Screenshot of Look -alike Application Implementation	56

Chapter 1

INTRODUCTION

Internships play a significant role in bridging the gap between academic knowledge and real-world industrial experience by providing students with practical exposure to modern technologies, software development practices, and professional work environments. As part of my internship, I worked on the development of a Web-Based Coffee Ordering and Management Platform, a full-stack café management and customer service system designed to digitalize and streamline café operations. The project was developed to address common challenges faced by traditional cafés such as manual table booking, delayed food service, lack of coordination between staff members, and inefficient payment handling.

The platform provides a centralized solution where customers can browse cafés, view menus, reserve tables for specific dates and times, pre-order food, make secure online payments, track order statuses in real time, manage profiles, download receipts, and interact with an AI-powered café assistant for support related to bookings, refunds, offers, café timings, and payments. This internship project provided me with an opportunity to work on real-world business requirements while enhancing my technical and problem-solving skills in full-stack web development.

The project supports multiple user roles including Admin, Café Owner, Chef, Waiter, Event Staff, and Customer, each with dedicated functionalities and role-based access control mechanisms. The admin manages café owner accounts and system-level operations, while café owners can manage café profiles, menus, tables, reservations, staff accounts, payments, event bookings, and customer orders through an interactive dashboard.

Chefs can monitor incoming food orders and update food preparation statuses, whereas waiters receive notifications for ready orders and update service statuses after delivering food to customers. Customers are required to complete profile verification and email authentication before accessing the platform, ensuring secure and authenticated usage of the application. The platform also incorporates real-time order flow management to ensure smooth coordination between customers and café staff, thereby reducing waiting time and improving the overall customer experience.

During the internship, I gained practical experience in designing and developing scalable web applications using modern technologies and development methodologies.

The backend system was developed using Spring Boot to manage APIs, authentication, order processing, reservations, and database operations efficiently. Spring Security with JWT authentication was implemented to provide secure access control and protect sensitive user data. MySQL was used for managing structured data, while React was utilized to build a responsive and user-friendly frontend interface for both customers and café staff.

Additional features such as online payment integration using Razorpay or Stripe, email verification through Java Mail Sender, and real-time updates using WebSocket technology were incorporated to improve the platform's functionality and performance. Through this internship project, I enhanced my understanding of full-stack development, database management, secure authentication mechanisms, API integration, and real-time workflow coordination while working on a practical solution for modern café operations and customer service management.

Furthermore, this internship provided me with valuable exposure to real-world software development practices, including system design, API development, database management, and frontend-backend integration. It allowed me to understand how large-scale applications are structured and how different modules work together to deliver a complete user experience. Working on this project helped me improve my problem-solving skills and gave me practical experience in converting business requirements into technical solutions.

I also gained confidence in debugging issues, optimizing performance, and following industry-standard coding practices. Overall, this experience significantly contributed to my technical growth and enhanced my ability to develop scalable and efficient full-stack web application.

1.1 Internship Overview

An internship forms one of the most essential components of an engineering student's academic journey, as it bridges the gap between theoretical learning and real-world industrial application. It provides students with valuable exposure to professional development environments, modern tools and technologies, software engineering practices, teamwork, and project execution methodologies. Such experiences help in understanding how real-time systems are designed, developed, and maintained in the industry, which cannot be fully achieved through classroom learning alone.

As part of my final semester B.Tech in Computer Science and Engineering, I completed my internship under the Infosys Springboard Virtual Internship 6.0 program. This internship provided a structured and industry-oriented learning experience focused on full-stack development and practical software engineering concepts. During this period, I worked on a Web-Based Coffee Ordering and Management Platform, a full-stack project designed to simulate real-world café operations and service management. The internship offered me the opportunity to apply my academic knowledge to a practical project involving multiple user roles, real-time workflows, secure authentication, and system integration. It helped me understand how scalable applications are designed to manage customers, staff, and business operations within a unified digital platform.

The internship was highly structured and included guided learning modules, project-based implementation, and continuous evaluation of progress. It started with foundational learning on web development concepts and gradually progressed toward building a complete full-stack application. I was responsible for implementing core functionalities such as user management, café booking, order processing, payment integration, and role-based access control.

Additionally, I gained experience in integrating frontend and backend systems, managing databases efficiently, and ensuring secure and optimized application performance. The collaborative learning environment and project-based approach of Infosys Springboard significantly enhanced my technical skills, problem-solving abilities, and overall understanding of real-world software development practices.

1.2 Company Profile

The Web-Based Coffee Ordering and Management Platform is a modern full-stack digital system designed to streamline and automate café operations by integrating customers, café owners, chefs, waiters, and event staff into a single unified platform. The system is built to address the limitations of traditional café management, such as manual table booking, delayed order processing, lack of coordination between staff members, and inefficient payment handling. It enhances customer experience through a user-friendly interface while enabling café management to efficiently oversee daily operations, analyze business performance, and improve overall service quality.



Fig 1.1: Company Logo

As you can see in Fig 1.1, this is the logo of our company which represents the core idea of the system-bringing together technology, service, hospitality into one integrated digital ecosystem. The logo symbolizes efficiency, connectivity, innovation in café management, highlighting the platform's goal of providing a smooth and modern dining experience.

The system enables customers to browse cafés, view menus, book tables, pre-order food, make secure online payments, track order status in real time, manage profiles, download receipts, and interact with an AI-powered café assistant for support regarding bookings, refunds, offers, café timings, and payment-related queries. It also includes a multilingual feature that allows users to interact with the system in their preferred language, making it more accessible, user-friendly for a diverse customer base.

The platform supports multiple user roles including Admin, Café Owner, Chef, Waiter, and Customer, each with well-defined responsibilities and role-based access control. Café owners manage café details, menus, tables, staff, and bookings, while chefs handle food preparation and update order statuses. Waiters are responsible for serving orders and managing delivery at tables, ensuring smooth coordination between kitchen and service operations. Customers interact with the system for browsing, ordering, booking, and payment processes, making the entire experience seamless and efficient. The system is designed to ensure real-time workflow management from order placement to final delivery.

Furthermore, the platform is developed using modern full-stack technologies and follows industry-standard software engineering practices. It emphasizes scalability, security, and performance through structured architecture, secure authentication mechanisms, and efficient database management. Working on this project provided practical exposure to real-world application development, including system design, API integration, and real-time data handling.

1.3 Training and Skill Development

An important component of my internship under the Infosys Springboard Virtual Internship 6.0 program was the structured training and skill development phase that supported my work on the Web-Based Coffee Ordering and Management Platform. This phase was designed to strengthen core technical knowledge and prepare interns to work on real-world full-stack development projects.

During this training, I enhanced my understanding of frontend development using React, where I learned to build dynamic, responsive, and user-friendly interfaces for customers and café staff. I worked on concepts such as component-based architecture, state management, props handling, and API integration to ensure smooth communication between frontend and backend systems. I also improved my skills in designing responsive layouts to ensure a consistent user experience across different devices. This helped me create interactive pages for café browsing, menu display, booking systems, and order tracking within the project.

On the backend side, I gained practical knowledge of Spring Boot, which was used to develop RESTful APIs for the café management system. I learned how to structure applications using layered architecture, implement controllers, services, and repositories, and manage business logic efficiently. Special focus was given to implementing secure authentication and authorization using Spring Security and JWT, ensuring safe access for different user roles such as customers, café owners, chefs, and waiters. This helped me understand how real-time enterprise applications manage security and user access control.

Additionally, I developed strong skills in database management using MySQL, where I learned to design relational schemas for entities such as users, cafés, menus, orders, bookings, and payments. I practiced writing efficient queries and maintaining data consistency across the system. I also gained exposure to API testing, debugging, and integrating frontend and backend modules to ensure smooth end-to-end functionality. Overall, this training phase significantly improved my technical proficiency and prepared me to successfully contribute to the development of a real-time café management platform. Furthermore, the training program enhanced my problem-solving, debugging, and project development skills through practical assignments and hands-on implementation.

1.4 Projects and Work Experience

Following the training phase, I transitioned into working on my internship project, the Web-Based Coffee Ordering and Management Platform, developed under the Infosys Springboard Virtual Internship 6.0 program. This project is a full-stack café management system designed to simulate real-world café operations, including customer ordering, table booking, staff coordination, payment processing, and real-time order tracking. My role in the project involved contributing to both backend and frontend development, which helped me gain practical exposure to end-to-end application development.

On the backend, I worked on developing and integrating RESTful APIs using Spring Boot to handle core functionalities such as user management, café operations, menu handling, bookings, orders, and payments. I contributed to designing and updating database entities for modules like User, Café, Table, Menu, Booking, Order, and Payment using MySQL. I also implemented and enhanced authentication mechanisms using Spring Security and JWT, ensuring secure access for different user roles such as customers, café owners, chefs, and waiters.

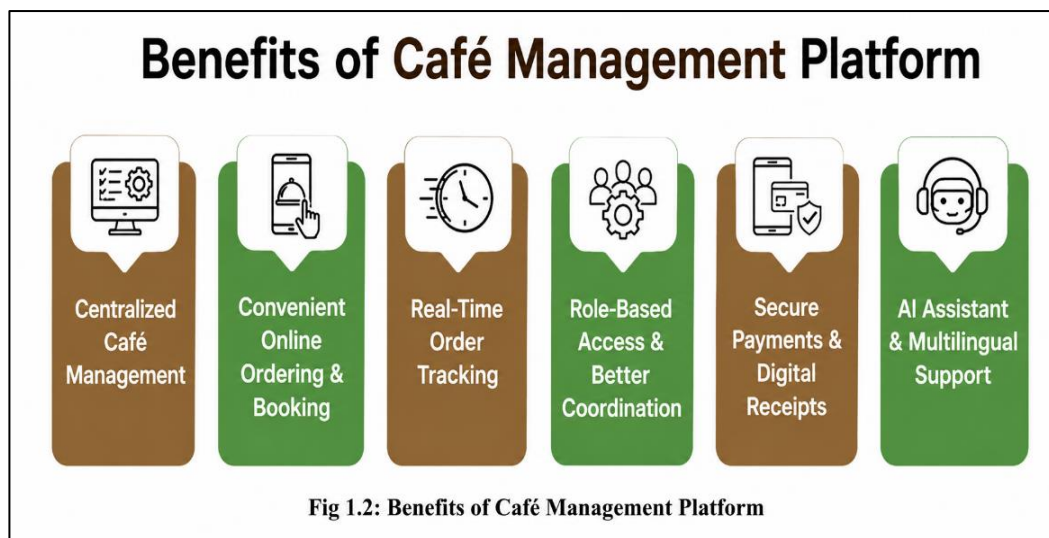


Fig 1.2: Benefits of Coffee Management Platform

Fig 1.2 shows the benefits of the Café Management Platform. The frontend work was equally engaging. Using React, I developed responsive and user-friendly interfaces that communicated with the backend APIs. I worked on modules such as café listing, menu display, booking, cart management, and real-time order tracking. I also improved dashboard usability, resolved UI and responsiveness issues, and contributed to

multilingual support and role-based interfaces to enhance overall user experience and application performance.

During development, I focused on effective state management and proper API integration to ensure seamless data flow between frontend and backend systems. I handled conditional rendering to improve usability and make the interface more interactive based on user roles and actions. Additionally, I worked on resolving UI and responsiveness issues to ensure the application performed well across different devices. This experience significantly improved my understanding of React component architecture, API communication, and building scalable and responsive web interfaces.

1.5 Learning Outcomes

The development of the Web-Based Café Ordering and Management Platform was a valuable learning experience that enhanced my technical and problem-solving skills. Through this project, I gained practical knowledge of full-stack web development using technologies like React.js, Spring Boot, MySQL, REST APIs, JWT Authentication, and WebSocket.

Overall, this project helped me build a strong foundation in software development, system design, database management, and secure application development. It improved my understanding of workflow management, API integration, and user experience while increasing my confidence in developing scalable and real-world business applications. The experience and skills gained from this project will be highly beneficial for my future career in web development and software engineering.

Additionally, the project provided practical exposure to real-world software development workflows such as debugging, version control, role-based access control, and collaborative development using Git and GitHub. I learned how to design responsive user interfaces, manage backend services efficiently, handle secure authentication and authorization, and implement dynamic features like real-time order tracking and notifications. Working on different modules of the application also improved my ability to analyze requirements, solve technical issues, optimize application performance, and maintain clean and scalable code architecture in a professional development environment.

1.6 Organization of the Report

This report is structured into five chapters:

Chapter 1: Introduction

This chapter provides an overview of the internship project, objectives, technologies used, and learning outcomes achieved during the internship. It introduces the Web-Based Café Ordering and Management Platform and explains the purpose of developing a full-stack application for café operations, customer management, table booking, event reservations, and online ordering.

Chapter 2: Company Background

This chapter describes Infosys Springboard, its vision, learning platform, and virtual internship opportunities provided to students. It explains how the internship program helped in developing technical skills through hands-on learning, industry-level assignments, and real-world project implementation. The chapter also discusses the learning resources, mentorship, and practical exposure provided during the internship journey.

Chapter 3: Technology Stack

This chapter highlights the technologies used in the project, such as React.js, Spring Boot, MySQL, JWT Authentication, and GitHub. It explains the role of each technology in frontend development, backend development, database management, authentication, version control, and real-time communication.

Chapter 4: Technical Training and Assignment

This chapter summarizes the development phase covering frontend and backend development, authentication, database operations, and API integration.

Chapter 5: Internship Project Contribution

This chapter explains the implementation of the café management platform, including booking, ordering, role-based access control, and real-time order tracking. It covers frontend enhancements, backend optimization, middleware implementation, report management, responsive.

Chapter 2

COMPANY BACKGROUND

Infosys Springboard is a digital learning and skill development initiative launched by Infosys to empower students and early professionals with industry-relevant knowledge and practical experience. It provides structured learning paths, free online courses, and virtual internship opportunities across domains such as software development, data structures, web technologies, and emerging IT skills. The platform is designed to bridge the gap between academic education and industry requirements by promoting hands-on, project-based learning and self-paced skill development. Through its structured modules, learners are encouraged to build real-world applications and strengthen their technical foundation.

From the very beginning, Infosys Springboard has focused on providing accessible and practical learning experiences that prepare learners for real industry challenges. The virtual internship program emphasizes project-based development, where students work on real-world problem statements to improve their technical and problem-solving abilities. It encourages learners to apply concepts such as frontend and backend development, database management, authentication, and system design in practical scenarios. This approach helps students gain industry exposure, improve coding skills, and build confidence for future career opportunities in the IT sector.

Infosys Springboard is a digital learning and skill development initiative launched by Infosys with the aim of empowering students and young professionals with industry-relevant knowledge and practical exposure. It provides a wide range of free learning resources, including structured courses, interactive content, and guided learning paths across domains such as programming, software development, data structures, web technologies, and emerging IT fields. The platform is designed to bridge the gap between academic education and industry expectations by promoting self-paced, flexible, and outcome-based learning. It encourages learners to build strong conceptual understanding along with practical skills through hands-on exercises and real-world projects.

The Infosys Springboard Virtual Internship program further enhances this learning experience by focusing on project-based and experiential learning. It allows students to apply theoretical concepts to real-world problem statements, thereby

strengthening their technical and problem-solving abilities. Through this internship, learners gain exposure to core software development practices such as frontend and backend development, database management, authentication, system design, and API integration. The program also helps in improving professional skills like logical thinking, debugging, and project implementation, ultimately preparing students for industry-level roles and future career opportunities in the IT sector.

2.1 Services Provided

Our Café Management and Customer Service Platform offer a complete set of digital services designed to improve café operations, customer experience, staff coordination, and secure online management. The platform combines customer-facing features, café owner management tools, staff services, AI-based assistance, and system security functionalities into a single integrated solution. The objective of the platform is not only to simplify café operations but also to provide a smart, secure, and user-friendly environment for customers, café owners, and employees. Through automation, real-time updates, and secure payment integration, the system helps cafés operate efficiently while improving customer satisfaction. One of the major services provided by the platform is customer service management. Customers can browse cafés, view menus, add food items to the cart, place online orders, book tables, reserve event packages, and make secure online payments. The platform also allows users to track their order status in real time, download payment receipts, request refunds, and manage their personal profiles. To improve accessibility and usability, the system supports multiple languages and includes an AI Café Assistant that helps users with booking queries, order tracking, cancellation requests, and café-related information.

These services provide customers with a smooth and convenient digital café experience. The platform also provides dedicated dashboards for café owners, chefs, waiters, and administrators to manage orders, bookings, menus, payments, and staff activities efficiently. These services improve coordination between users, reduce manual work, and help maintain smooth daily café operations through a centralized digital system. The platform offers real-time notifications for order updates, booking confirmations, and payment status to improve communication between customers and café staff. The integrated dashboard provides analytical insights into sales, customer activities, and operational performance, helping café owners make informed business decisions.



Fig 2.1: Services Provided

Fig 2.1 shows all services provided by the Café Management and Customer Service Platform. In addition to customer services, the platform also provides comprehensive café owner services. Café owners can manage café profiles, update menus and food items, monitor table availability, handle reservations, manage customer orders, and oversee online payments. The system also supports event booking management, staff management for chefs, waiters, and event coordinators, as well as offer and discount management. Furthermore, café owners can access reports and analytics to monitor sales, customer preferences, booking trends, and overall business performance. These features help café owners efficiently manage daily operations and make data-driven business decisions.

The platform also includes dedicated staff services that help improve coordination and workflow inside the café. Staff members can view assigned tasks, track order progress, update order status, and manage kitchen or delivery-related activities. Chefs can monitor kitchen orders, waiters can handle customer service and delivery management, while event staff can coordinate event setups and bookings. Real-time notifications ensure smooth communication between customers, café owners, and staff members. Additionally, the system stores work history and performance records, enabling better workforce management efficiency.

Another important aspect of the platform is the AI Café Assistant and system security services. The AI assistant provides instant support for booking queries, payment assistance, refund and cancellation support, café timings, location details, offers, discounts, and general café-related questions. Along with smart assistance, the platform emphasizes strong security mechanisms such as secure user authentication, role-based access control, encrypted payment processing, data protection, audit logging, backup and recovery management, and regular security updates. The system also supports real-time performance monitoring and multi-language accessibility, ensuring a secure, reliable, and scalable café management environment for all users.

2.2 Vision, Mission & Certifications

At the heart of Infosys Springboard Virtual Internship 6.0 lies a clear and inspiring vision: to empower students and learners by providing industry-relevant virtual learning experiences that enhance technical knowledge, practical skills, and problem-solving abilities. The program believes that learning should not remain limited to theoretical concepts alone. Instead, it should provide real-world exposure and measurable value by helping learners apply their knowledge through projects, assignments, and hands-on activities. This vision drives every aspect of the internship program, from content delivery and technical training to learner engagement and career development.

Complementing this vision is the mission of Infosys Springboard: to bridge the gap between academic learning and industry expectations by offering accessible, flexible, and skill-oriented virtual internship opportunities. The internship focuses on delivering practical learning experiences in modern technologies and software development practices.

Through guided learning modules, coding tasks, assessments, and project-based activities, the program helps learners improve technical expertise, communication skills, and professional confidence. Rather than focusing only on theoretical education, Infosys Springboard aims to prepare students for real-world industry challenges and career opportunities. The internship program also encourages continuous learning, teamwork, and problem-solving through hands-on project development and real-time implementation tasks.

To reinforce its commitment to quality and learning excellence, Infosys follows globally recognized standards and best practices in training and technology development. Infosys Limited is certified under ISO 9001:2015 for Quality

Management Systems and ISO 27001:2013 for Information Security Management Systems (ISMS). These certifications ensure that the organization maintains high standards in quality, security, process management, and data protection. Through secure digital learning platforms and structured learning methodologies, Infosys ensures a reliable and effective learning environment for students and professionals.

These quality standards and security practices reflect Infosys' strong commitment to operational excellence, innovation, and learner trust. Participants of Infosys Springboard can confidently rely on the platform for secure access to learning resources, accurate training content, and industry-focused educational experiences. The program's learner-centric approach, combined with modern technologies, real-time learning resources, and practical project exposure, helps students improve their employability and technical capabilities. These practices also enable Infosys to support learners from diverse educational backgrounds and skill levels effectively.



Fig 2.2: Company Clients

Fig 2.2 shows the learners and industry collaborations associated with Infosys Springboard Virtual Internship 6.0. In essence, the program's vision, mission, and certifications reflect its commitment to providing quality digital learning and industry-focused skill development. Infosys follows globally recognized standards such as ISO 9001:2015 and ISO 27001:2013, ensuring quality management and secure learning practices. Through practical training, project-based learning, and continuous innovation, Infosys Springboard continues to empower students and develop future-ready professionals. The Coffee Ordering and Management Platform serves customers, café owners, administrators, chefs, and waiters by providing a centralized digital environment for managing café operations.

2.3 Community Engagement & SB Meetups

Community engagement and continuous learning are important parts of Infosys Springboard Virtual Internship 6.0 and its long-term vision. The program strongly believes that learning should not be limited only to classrooms, but should also encourage collaboration, practical exposure, and knowledge sharing among students and professionals. As technology continues to evolve rapidly, learners need opportunities to improve their technical and professional skills through real-world learning experiences. Infosys Springboard actively contributes to student development by providing virtual internships, training programs, certifications, and industry-oriented learning resources that help learners become more prepared for future career opportunities.

Beyond technical learning, the Infosys Springboard platform also acts as a career development and talent-building initiative for students and young professionals. Participants get opportunities to improve their technical knowledge, communication skills, problem-solving abilities, and project development experience through hands-on activities and guided learning modules. The platform also helps learners connect with industry practices, gain career guidance, and explore future opportunities in software development and emerging technologies. Through these initiatives, Infosys Springboard supports learners in building confidence and preparing for real-world industry environments.

Ultimately, Infosys views its learning and community initiatives not only as educational programs but also as a responsibility toward empowering future professionals. By improving digital learning accessibility and promoting skill development, the organization contributes to building a stronger and more knowledgeable technology community. The success of Infosys Springboard has encouraged the organization to continue expanding its learning ecosystem through certifications, virtual training, project-based learning, and collaborations with educational institutions.

As part of the internship journey, community engagement activities and SB Meetups provided learners with opportunities to interact with mentors, industry experts, and fellow participants from different technical backgrounds. These meetups encouraged knowledge sharing, collaborative discussions, and practical exposure to current industry trends and technologies. Through virtual sessions and interactive

events, students were able to learn from real-world experiences shared by professionals working in the software development industry.

The SB Meetups also created a platform where learners could discuss project ideas, development challenges, and innovative solutions related to full-stack web development, cloud technologies, artificial intelligence, and emerging software tools. These interactions helped improve communication skills, teamwork, and confidence while working in collaborative environments. It also allowed participants to understand how industry professionals approach problem-solving, project planning, and software development workflows in real-world organizations.

Another important benefit of these community activities was networking and professional growth. Students got opportunities to connect with peers, mentors, and technical communities, which helped in building professional relationships and expanding learning beyond the internship curriculum. Participation in these sessions improved awareness of industry expectations, career opportunities, and continuous skill development. Overall, the community engagement initiatives and SB Meetups played an important role in enhancing both technical and professional learning throughout the internship experience.

Throughout the internship, learners also gained valuable experience by working on real-world projects such as the Coffee Ordering and Management Platform, where they applied concepts of full-stack web development, database management, API integration, and software engineering practices. These project-based activities helped participants strengthen their practical implementation skills, understand industry-standard development workflows, and gain confidence in designing scalable and user-friendly software solutions. Such hands-on experiences bridged the gap between theoretical learning and real-world application, making the internship more impactful and career-oriented.

The internship also provided learners with opportunities to develop practical projects and apply their technical knowledge in real-world scenarios. Through hands-on implementation, collaborative learning, and continuous guidance from mentors, participants enhanced their problem-solving abilities, technical expertise, and understanding of industry-standard software development practices.

Chapter 3

TECHNOLOGY STACK

In any software development project, selecting the right technology stack is an important decision that directly affects the performance, scalability, security, and maintainability of the application. The term “technology stack” refers to the combination of programming languages, frameworks, databases, libraries, and tools used to design, develop, and deploy a software application. A properly selected technology stack helps developers build efficient, secure, and user-friendly applications while simplifying future maintenance and upgrades.

During the development of the Web-Based Coffee Ordering and Management Platform, a modern and efficient technology stack was used to build a complete full-stack application. The project uses React.js for frontend development, Spring Boot for backend services, MySQL for database management, and additional tools such as Spring Security, JWT authentication, Razorpay integration, GitHub, and WebSocket support. These technologies were selected because of their reliability, scalability, security, and suitability for developing real-world enterprise applications.

The combination of these technologies helped in building a centralized café management platform that supports multiple users and multiple cafés within a single system. React.js was used to create a dynamic, responsive, and user-friendly frontend interface for customers, café owners, chefs, waiters, and administrators. Spring Boot handled backend operations such as REST API development, authentication, role-based access control, booking management, payment processing, and business logic implementation. MySQL was used to store and manage application data such as users, café details, menus, orders, bookings, tables, payments, and staff information.

In addition to simplifying development, the selected technology stack supported modular and secure application architecture. Spring Security and JWT authentication were implemented to provide secure login and role-based authorization. WebSocket technology enabled real-time order status updates between chefs, waiters, and customers. Razorpay integration was used to handle secure online payments, branches, pull requests, and issue tracking. Collectively, this well-integrated stack provided a stable foundation that supported both rapid development and long-term maintainability of the application.

Overall, the technology stack played a vital role in the successful implementation of the project. It helped create a secure, scalable, responsive, and user-friendly café management platform capable of handling real-time operations and multiple user roles. The following sections describe the technologies used in the project, their key features, and how they contributed to the development of the application.

3.1 React.js

React.js is an open-source JavaScript library developed by Meta for building modern and interactive user interfaces. It allows developers to create reusable UI components and develop fast, responsive, and scalable web applications. React uses a component-based architecture and Virtual DOM, which improves rendering performance and provides a smooth user experience.

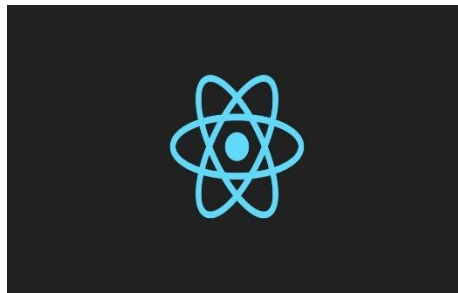


Fig 3.1: React.js

Fig 3.1 shows React.js. In this project, React.js was used to develop the frontend interface for customers, café owners, chefs, waiters, and administrators. It helped in creating responsive pages for café browsing, menu management, online ordering, table booking, payment processing, event booking, and dashboard management. React.js improved the user experience by enabling fast page rendering and smooth interaction without reloading the entire webpage.

React.js also allowed the application to be built using reusable components, which helped maintain consistency across different modules and reduced code duplication. Features such as dynamic routing, state management, conditional rendering, and API integration were implemented to provide real-time updates for orders, bookings, payments, and dashboard activities. Additionally, React Hooks such as `useState` and `useEffect` were utilized to manage application data and user interactions efficiently. The component-based architecture improved scalability, maintainability, and overall frontend performance, making the platform easier to update and enhance with new features in the future.

3.1.1 Key Features of React.js

React.js follows a component-based architecture that allows developers to create reusable and independent UI components. In this project, components such as navigation bars, booking forms, menu cards, dashboards, and payment sections were reused across different pages. This reduced code duplication, improved maintainability, and made the frontend development process more organized and efficient.

React.js uses a Virtual DOM mechanism that updates only the required parts of the webpage instead of reloading the complete page. This improves application performance and provides a faster and smoother user experience during operations such as menu browsing, order placement, booking management, and dashboard updates.

React.js provides fast rendering capabilities that make the application highly responsive and interactive. The framework efficiently handles dynamic content updates, making it suitable for real-time applications like café ordering, table booking, event reservation, and order tracking systems. React.js supports state management using hooks such as `useState` and `useEffect`. These hooks were used in the project to manage dynamic data like customer orders, cart items, booking details, payment status, notifications, and user sessions. This helped maintain smooth interaction between the frontend and backend systems.

React Router was used to implement navigation between different pages such as Home, Menu, Booking, Orders, Payments, Event Management, and Admin Dashboard without refreshing the application. React.js supports responsive user interface development, allowing the platform to work efficiently on desktops, tablets, and mobile devices. Responsive layouts and flexible UI components ensured better accessibility and usability for customers, café owners, chefs, waiters, and administrators.

3.1.2 Use Case in My Internship

During the development of the Web-Based Coffee Ordering and Management Platform, React.js formed the foundation for the frontend development of the application. The platform required both customer-facing pages and role-based dashboards for Admins, Café Owners, Chefs, Waiters, and Customers. Here is how I applied React.js features in real project scenarios.

I used React Router to create dynamic navigation between pages such as café details, menu pages, booking pages, order tracking, payment pages, and dashboards. JWT authentication and protected routes were implemented to provide secure access control for Admins, Café Owners, Chefs, Waiters, and Customers. React.js frontend was

integrated with Spring Boot REST APIs to manage data related to cafés, menus, orders, bookings, users, and payments.

Responsive designs were developed using React.js and CSS to ensure smooth functionality on desktop, tablet, and mobile devices. React hooks such as `useState` and `useEffect` were used to manage dynamic data like cart items, bookings, order status, notifications, and user sessions. I implemented language translation functionality to allow users to interact with the platform in their preferred language, improving accessibility and user experience.

3.2 Spring Boot

Spring Boot is a powerful Java-based framework used for developing secure, scalable, and enterprise-level web applications. It simplifies backend development by providing built-in configurations, dependency management, REST API support, and integration with databases and security frameworks. Spring Boot reduces development complexity and allows developers to create production-ready applications efficiently. In this project, Spring Boot was used as the backend framework to manage APIs, authentication, café operations, booking systems, payment processing, and database communication.

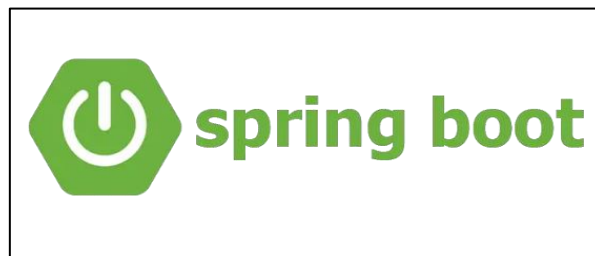


Fig 3.2: Spring Boot

Fig 3.2 shows Spring Boot. Spring Boot provided the backbone for the entire backend system of the café management platform. It handled communication between the frontend developed using React.js and the MySQL database. The framework also supported secure authentication, role-based access control, online payments, order management, event booking, and real-time application workflows. Its modular architecture and easy integration capabilities helped in building a secure and maintainable backend system.

3.2.1 Key Features of Spring Boot

Spring Boot provides auto-configuration features that automatically configure application components based on project dependencies. This reduces manual configuration work and speeds up backend development. Spring Boot simplifies the

creation of REST APIs using annotations like `@RestController`, `@GetMapping`, `@PostMapping`, and `@RequestMapping`. These APIs were used for managing users, cafés, bookings, orders, payments, and event reservations. Spring Boot uses Maven dependencies to manage external libraries and frameworks required for the application. This helps in maintaining project consistency and simplifies package management. Spring Boot supports secure authentication and authorization using Spring Security and JWT. It helps protect APIs, manage user roles, and restrict unauthorized access to admin and staff functionalities.

3.2.2 Application Workflow in My Internship

During the development of the Web-Based Coffee Ordering and Management Platform, Spring Boot was used as the core backend framework to manage APIs, authentication, database communication, and business logic. It served as the backbone of the system by connecting the React frontend with the MySQL database and ensuring smooth communication between customers, café owners, chefs, waiters, and administrators. Spring Boot was used to create RESTful APIs for modules such as user management, café management, menu handling, table booking, food ordering, payments, refunds, and event reservations. These APIs allowed smooth data exchange between frontend and backend.

Spring Security and JWT authentication were implemented to provide secure login and role-based access for Admin, Café Owner, Chef, Waiter, Event Staff, and Customer modules. The backend handled secure authentication using encrypted passwords and JWT tokens. Protected APIs were accessible only to authenticated users based on their assigned roles.

To maintain a clean, scalable, and organized backend structure, the project followed the Model-View-Controller (MVC) architecture. This architecture helped separate different functionalities of the application, making the system easier to manage, maintain, and expand in the future. The Model layer defined entities such as User, Cafe, Menu, Booking, Order, and Payment for database management. The Controller layer handled API requests and responses for different modules of the system. The Service layer managed business logic including booking validation, order processing, refund handling, and payment management, while the Repository layer performed database operations using Spring Data JPA.

This separation of concerns improved code maintainability, scalability, readability, and overall system management. Spring Data JPA and Hibernate were used for database communication and ORM (Object Relational Mapping). The backend managed entities such as Users, Cafés, Tables, Orders, Bookings, Payments, Events, and Refunds efficiently. These technologies simplified CRUD operations, reduced manual SQL queries, and maintained relationships between tables. Spring Boot validations were implemented to ensure proper user input, secure data processing, and reliable system behaviour. Exception handling mechanisms were used to manage errors and provide meaningful responses. The backend managed order status updates such as Order Placed → Preparing → Ready → Served, helping chefs and waiters coordinate efficiently within the café system.

The backend also managed cancellation requests and refund processing based on booking and order policies. Refund status and payment records were updated automatically within the system. Separate backend functionalities were developed for Admin, Café Owner, Chef, Waiter, Event Staff, and Customer dashboards, allowing each user to access only their authorized operations.

3.3 MySQL

MySQL is a widely used open-source relational database management system that stores application data in a structured format using tables. It is known for its reliability, security, scalability, and high performance, making it one of the most popular databases for web-based applications. MySQL helps manage large amounts of data efficiently while maintaining data consistency and integrity. Due to its strong support for relational data handling and smooth integration with backend technologies like Spring Boot and Hibernate, it was used as the primary database system in this project.



Fig 3.3: MySQL

Fig 3.3 shows MySQL. In this project, MySQL was used to store and manage customer details, café information, menus, table bookings, food orders, payments, staff details, event reservations, refund records, and order tracking data. It worked together with Spring Boot, Spring Data JPA, and Hibernate to provide secure and efficient database communication for the café management platform.

3.3.1 Key Features of MySQL

MySQL stores data in tables with proper relationships between entities such as Users, Cafés, Orders, Tables, Payments, and Bookings. This relational structure helps maintain organized and consistent application data. MySQL provides secure authentication, user access management, and role-based permissions to protect sensitive customer and payment information from unauthorized access.

MySQL supports fast query execution, indexing, and optimized data retrieval, helping the application manage bookings, orders, and payments efficiently even with multiple users. It also maintains data integrity and ensures smooth performance for large-scale café management systems. Additionally, it provides scalability, security, and reliable data storage, which makes it suitable for handling increasing users and growing business data.

MySQL can handle large amounts of application data and multiple concurrent users, making it suitable for growing café businesses and multi-café management systems. It also ensures fast data retrieval and reliable performance even under heavy load. MySQL supports backup and recovery mechanisms that help protect important business data and ensure system reliability during unexpected failures. It also ensures data consistency and supports secure transaction management, which is important for handling orders, bookings, payments, and customer records in the café management system.

MySQL integrates smoothly with Spring Boot, Hibernate, and Spring Data JPA, simplifying database operations and reducing development complexity. It also improves data management efficiency by supporting easy entity mapping, query handling, and seamless communication between the backend application and the database.

3.3.2 Application Workflow in My Internship

In this project, MySQL served as the central database system for storing and managing all application data. It worked together with Spring Boot and Hibernate to support secure and efficient backend operations.

- I designed and managed relational tables in MySQL for core entities such as User, Client, Payments & Refunds, Orders & Bookings, and Menu.
- User included fields like name, email, password hash, role, and account status.
- Client included café name, location, timings, and owner details.
- Payments & Refunds were used to store transaction details and refund status.
- Orders & Bookings were used to track food orders, table reservations, and event bookings.
- Menu included item name, price, category, and seasonal offers.

These tables were structured using primary keys, foreign keys, and constraints to maintain data integrity and consistency.

MySQL supported all Create, Read, Update, Delete operations through Spring Boot APIs and Hibernate ORM. This included adding new cafés, menu items, and users, fetching orders, bookings, and payment details, deleting or cancelling bookings and orders when required, and updating order status such as Placed → Preparing → Ready → Served. I implemented SQL queries to support filtering and data retrieval such as fetching all menu items for a specific café, retrieving orders based on status like Pending, Preparing, and Completed, filtering bookings by date, time, and user, and getting payment history for a specific customer. MySQL queries ensured fast and accurate data retrieval for dashboard and user operations.

MySQL supported relational data structure using foreign keys. Each order was linked to a user and a café, each menu item belonged to a specific café, and each booking was connected to a customer and a table. This relationship design ensured data consistency and helped avoid duplication of data across different tables, making the database more organized, reliable, and easier to maintain.

Role information was stored in the MySQL database and used in backend logic to control access across the system. Based on assigned roles, different users were given different permissions. For example, Admin can access all café and user data, café owners can manage only their own cafés, customers can view only their orders and bookings, and staff can access only their assigned tasks. This ensured proper role-based access control and improved system security. This ensured secure and role-based access control across the system, improving data security and user privacy.

Chapter 4

TECHNICAL TRAINING AND ASSIGNMENT

The initial phase of my internship journey was centred around an intensive and well-structured technical training program that helped build a strong foundation in full-stack web development. The training focused on both frontend and backend technologies used in real-world applications. I started learning React.js fundamentals such as HTML, CSS, JavaScript, JSX, components, props, hooks, state management, routing, API integration, forms, and error handling. This training helped me understand how modern responsive web applications are designed and developed efficiently.

The next phase of training focused on advanced React development using Vite and backend development using Spring Boot. I learned Java and Object-Oriented Programming concepts, REST API development, Controllers, Services, Models, Spring Data JPA, and MySQL database integration. This phase helped me understand how frontend and backend communicate with each other to create complete web applications. I also learned how APIs manage data flow between the user interface and the database in a structured and secure manner.

Another important aspect of the training was learning various development tools and best practices used in professional software development. Tools such as Postman were used for API testing, GitHub was used for version control and project management, and Maven was used for dependency management and project build automation. In addition, I worked with integrations like Razorpay for online payments and Google Translate for multi-language support. This hands-on exposure helped me understand the practical workflow followed in industry-level application development.

Furthermore, the training program included practical assignments and mini-projects that strengthened my understanding of software development concepts. These assignments required implementing frontend interfaces, developing backend APIs, connecting databases, and testing application functionalities. Through continuous practice and problem-solving activities, I improved my coding skills, debugging techniques, project development experience, and ability to build complete full-stack web applications following industry standards. The technical training provided valuable industry exposure and enhanced my confidence in developing scalable, secure, and user-friendly web applications independently.

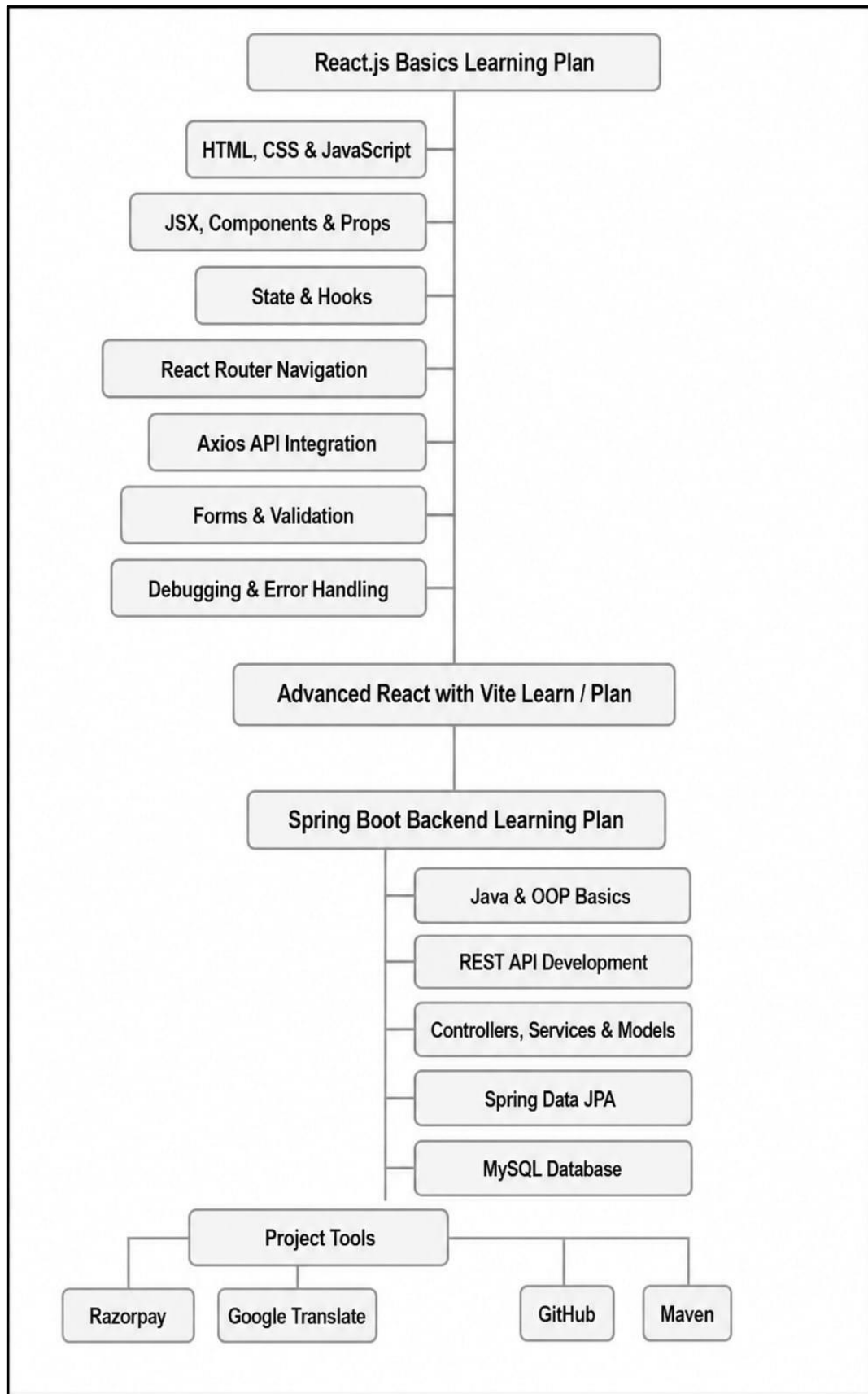


Fig 4.1: Training Program Flow

To apply all these concepts together, I worked on a real-time Web-Based Coffee Ordering and Management Platform as my major project. In this project, I implemented customer ordering, café management, table booking, online payments, multilingual support, AI chatbot assistance, and role-based dashboards for Admin, Café Owner, Chef, and Waiter. The project allowed me to practically implement frontend-backend integration, REST APIs, database operations, authentication, and real-world business logic. Overall, this training and project development phase greatly improved my technical skills and prepared me for professional software development environments.

4.1 React Fundamentals and Advanced Concepts

React.js was one of the main frontend technologies used in the development of my Web-Based Coffee Ordering and Management Platform. During the project development phase, I learned important React fundamentals such as JSX, components, props, state management, event handling, forms, and React Router for page navigation. These concepts helped me build interactive and responsive user interfaces for customer dashboards, café management pages, booking systems, menu sections, and payment modules. I also learned how frontend applications communicate with backend APIs using Axios and Fetch API.

After understanding the basics, I worked on advanced React concepts such as React Hooks (`useState` and `useEffect`), reusable components, API integration, asynchronous operations, error handling, and frontend optimization. These concepts helped me develop scalable and maintainable frontend modules for food ordering, table booking, event reservation, seasonal offers, multilingual support, and AI chatbot functionality.

4.1.1 Core React Concepts

At the fundamental level, I learned the core concepts of React.js such as components, JSX, props, and state management. I understood how React uses reusable components to build scalable user interfaces and how props are used to transfer data between components. I also learned state handling using React Hooks like `useState`, which helped in creating dynamic and interactive frontend features for the café management platform.

The training also covered important frontend concepts such as event handling, conditional rendering, form handling, and list rendering. Through practical implementation, I worked on customer forms, menu displays, booking systems, cart functionality, and dashboard interfaces.

Control flow and dynamic UI rendering were practiced extensively during frontend development. I implemented features such as showing booking details based on user actions, filtering menu items, displaying order status, and handling validations using React logic. React Router was also used for navigation between different pages such as login, customer dashboard, café details, booking pages, and payment sections.

One of the most important learning experiences was understanding React Hooks and reusable component structures. I learned the use of `useState` and `useEffect` for handling application state and side effects. These concepts helped me build maintainable frontend modules for food ordering, table booking, multilingual support, seasonal offers, refund management, and AI chatbot functionality in the project.

4.1.2 Asynchronous Operations and API Handling in React

After understanding the core concepts of React.js, I learned how asynchronous operations work in frontend applications, especially while handling backend API communication. Since the café management platform depended on real-time data such as menus, bookings, orders, payments, and customer details, API integration became an important part of development. I used `Fetch API` and `Axios` to exchange data between the React frontend and Spring Boot backend, which helped in creating dynamic and responsive application features.

I also learned how to work with asynchronous programming using `async/await` and `Promise` handling for managing API requests efficiently. Error handling using `try-catch` blocks was implemented to manage failed requests, invalid inputs, and server-side errors properly. These concepts improved the reliability and stability of modules such as food ordering, table booking, payment processing, refund handling, and event reservation.

Another important concept I worked with was React Hooks, especially `useEffect`, for handling side effects like API calls and automatic data loading. This helped the frontend update data dynamically whenever customer actions or backend responses changed. Features such as order tracking, cart updates, menu filtering, and booking status updates depended heavily on asynchronous data handling.

I also gained practical understanding of frontend execution flow, component rendering, and API response handling during development. Debugging asynchronous issues using browser developer tools, console logs, and API testing helped me improve application performance and responsiveness.

4.1.3 React Features and Advanced Frontend Practices

The project development also focused on modern React features and frontend practices that helped make the application more dynamic, scalable, and maintainable. Features such as JSX, reusable components, and props were used to build clean and modular user interfaces. I also worked with dynamic rendering techniques for displaying menus, bookings, cart items, event details, and customer information efficiently across different pages of the café management platform.

React Hooks such as `useState` and `useEffect` were used extensively for state management and handling side effects like API calls and automatic data updates. I also implemented conditional rendering, form validation, optional chaining, and dynamic data handling to improve application stability and user experience. These concepts helped create responsive frontend modules for food ordering, table booking, payments, multilingual support, and AI chatbot functionality.

Advanced array methods such as `map()`, `filter()`, `find()`, and `forEach()` were used regularly for handling menu data, filtering café lists, displaying booking records, and managing customer orders. These methods helped simplify frontend logic and made the code more readable and efficient. Real-world frontend features like search filters, category-based menus, order tracking, and dashboard listings were developed using these concepts.

Another important area of learning was frontend optimization and reusable component architecture. I worked on creating reusable UI components to reduce code duplication and improve maintainability. Debugging techniques, API error handling, and performance optimization methods were also practiced during development. These concepts helped improve application responsiveness and provided a better user experience across the entire café management platform.

Towards the end of the development process, I also focused on frontend debugging and performance optimization techniques. Browser developer tools, API debugging, error handling methods, and optimized rendering practices were used to improve application responsiveness and user experience. These concepts helped me build a stable, efficient, and scalable frontend for the café management platform.

4.2 Backend Development with Spring Boot

Backend development played an important role in my Web-Based Coffee Ordering and Management Platform because it handled business logic, authentication, database communication, and API management. Using Spring Boot, I developed backend modules for user management, café management, food ordering, table booking, payments, refunds, and event reservations. This helped me understand how backend systems process requests and connect the React frontend with the MySQL database efficiently.

The project also included secure authentication and role-based access control using Spring Security and JWT authentication. Spring Data JPA and Hibernate were used for database operations, while Controllers, Services, and Repositories helped maintain a structured backend architecture. Overall, this provided practical experience in building secure and scalable web applications.

4.2.1 Setting Up the Development Environment and Exploring Spring Boot

My development journey began with setting up the Spring Boot environment using Java, Maven, and IntelliJ IDEA/VS Code. I learned how to create Spring Boot projects, manage dependencies using Maven, configure application properties, and organize project structure efficiently. I also explored important backend concepts such as Controllers, Services, Repositories, and Entity classes.

I also gained understanding of backend request handling, REST APIs, and asynchronous communication between the React frontend and Spring Boot backend. Features such as API requests, database operations, authentication handling, and dynamic data loading helped me understand how backend systems process multiple user requests efficiently. Working with APIs for food ordering, bookings, payments, and customer management improved my practical understanding of real-world backend workflows.

To strengthen these concepts, I implemented backend modules involving database communication, API integration, authentication, and CRUD operations. I worked with Spring Data JPA, Hibernate, MySQL, and JWT authentication to handle secure data management and role-based access control. This practical implementation helped me develop clean, organized, and scalable backend logic for the café management platform.

4.2.2 RESTful API Development with Spring Boot

After gaining a strong understanding of Spring Boot fundamentals, I started developing structured RESTful APIs for different modules of the café management system. Spring Boot simplified backend development through annotations, dependency injection, and built-in configurations. I created APIs using HTTP methods such as GET, POST, PUT, and DELETE for handling users, cafés, menu items, orders, bookings, payments, refunds, and event reservations.

Each API was designed to handle request parameters, request bodies, validations, and proper response handling. I implemented features such as JSON data processing, exception handling, authentication checks, and secure endpoint access using Spring Security and JWT authentication. Service classes and reusable methods helped maintain clean and modular backend logic throughout the project.

The project also followed the MVC architecture to organize the backend into Controllers, Services, Repositories, and Entity classes. Controllers handled API requests, Services managed business logic, and Repositories performed database operations using Spring Data JPA and Hibernate. This structured approach made the application easier to maintain, debug, and extend with additional features.

Database integration was handled using MySQL along with Spring Data JPA and Hibernate for efficient data persistence and management. Relationships between entities such as users, cafés, bookings, orders, payments, and menu items were properly mapped to maintain structured data flow within the application.

During project development, I implemented modules such as customer registration, login, café management, menu handling, table booking, food ordering, event booking, payments, and refunds. These features helped me understand API workflow, authentication, database communication, response handling, and error management in a real-world application.

4.2.3 Integrating MySQL and Spring Data JPA for Data Persistence

A major part of backend development in my project was database management, where I used MySQL along with Spring Data JPA and Hibernate for storing and managing application data. These technologies simplified database communication and backend data handling efficiently. I created Entity classes and tables for users, cafés, orders, bookings, payments, and menu management. I learned to define entity relationships and mappings, apply validations and constraints, use automatic table generation and timestamps, and manage database operations through repositories.

CRUD operations were performed using JPA repository methods such as `save()`, `findById()`, `findAll()`, and `deleteById()`. I also implemented entity relationships like One-to-Many and Many-to-One mappings, which helped me understand relational database management and backend data handling in real-world applications.

4.2.4 Implementing JWT Authentication and Authorization

Security was an important part of backend development in my project. I implemented JWT authentication using Spring Security to provide secure login and protected API access for different users in the system. The authentication process included accepting user credentials through the login API, verifying user details and generating a JWT token, sending the token to the frontend for future authenticated requests, and validating and decoding the token through security filters before accessing protected APIs.

I also implemented role-based access control (RBAC) by assigning roles such as Admin, Café Owner, Chef, Waiter, and Customer. Access to APIs and dashboards was controlled based on user roles and permissions. For example, café owners could manage only their cafés, while customers could access only their bookings and orders. I also implemented password encryption using `BCryptPasswordEncoder`, secure API access using Spring Security filters, protected routes for authenticated users only, and input validation and exception handling.

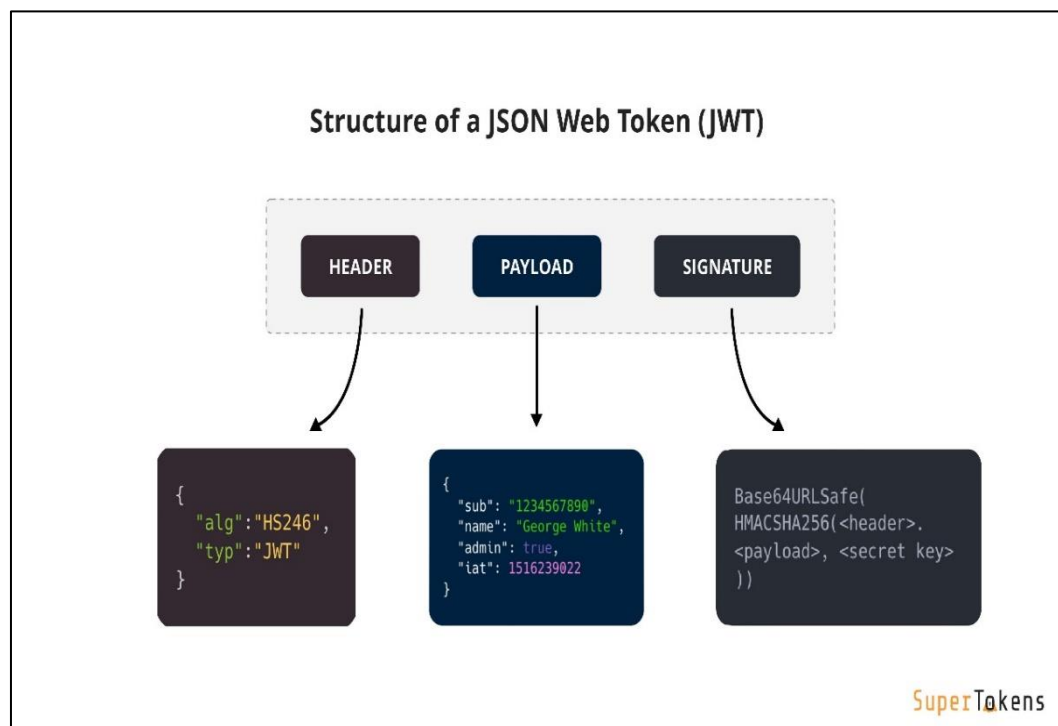


Fig 4.2: JSON Web Token

Fig 4.2 shows the JWT authentication workflow used in the application through Header, Payload, and Signature sections. These components help provide secure communication between the frontend and backend. This implementation improved the security and reliability of the backend APIs.

4.3 Database Operations with MySQL

One of the most important parts of full-stack web development in my project was managing and storing application data efficiently. During the development of the Web-Based Coffee Ordering and Management Platform, I worked extensively with MySQL, a widely used relational database management system known for its reliability, structured data storage, and strong performance. MySQL was used because it provides secure data handling, supports relationships between tables, and integrates smoothly with Spring Boot, Spring Data JPA, and Hibernate. Through project development, I learned how to design database tables, manage relationships, and handle real-world application data efficiently.

MySQL stored application data related to users, cafés, menu items, orders, bookings, payments, events, and refunds in a structured tabular format. Using Spring Data JPA and Hibernate, I created Entity classes, managed database relationships, and performed CRUD operations efficiently. Features such as validations, constraints, indexing, and relational mappings helped maintain data consistency and improve application performance. This practical implementation helped me understand real-world database management and backend data handling in enterprise-level web applications.

4.3.1 Database Design and Entity Modelling with MySQL

One of the initial tasks while working with MySQL was understanding database design and table relationships. I learned how to create structured database tables for entities such as Users, Cafés, Menu Items, Orders, Payments, Bookings, and Events. These tables included appropriate data types, constraints, default values, and relational mappings to maintain proper data organization and consistency.

For example, in the café management platform, I designed User tables containing customer and staff details, while Order and Booking tables were linked with users and cafés using foreign keys. This relational structure helped maintain accurate connections between different modules of the system. Using Spring Data JPA and Hibernate, I also implemented validations, entity mappings, and relationship

annotations, which improved database management and backend data flow throughout the application.

I also gained practical experience in designing normalized database schemas to reduce data redundancy and improve storage efficiency. Different tables were connected using one-to-one, one-to-many, and many-to-many relationships based on application requirements. Proper indexing and constraints were implemented to improve query performance and maintain data integrity while handling large amounts of customer, booking, and order-related information within the system.

Another important learning experience was understanding how database relationships support real-world business workflows in a full-stack application. For example, menu items were connected with cafés, payments were associated with bookings and orders, and customer profiles were linked with reservation history. These relational mappings helped maintain accurate data flow between frontend and backend modules while simplifying data retrieval and management operations.

Additionally, I worked on implementing validation rules and secure data handling practices within the database layer. Fields such as email addresses, mobile numbers, booking details, and payment information were validated to prevent invalid data entries. Hibernate annotations and JPA entity configurations helped automate database operations and improved communication between the Spring Boot backend and MySQL database. This practical implementation strengthened my understanding of relational database design, backend architecture, and enterprise-level data management systems.

Additionally, I learned how to optimize database queries and manage backend data efficiently for better application performance. Query methods, joins, filtering operations, and sorting techniques were used to retrieve customer details, café listings, booking history, and order information quickly from the MySQL database. This experience helped me understand how efficient database design and optimized query handling contribute to faster response times, improved scalability, and smooth functioning of enterprise-level web applications.

I also understood the importance of maintaining data consistency and secure data storage while managing multiple modules within the café management system. This experience improved my practical knowledge of relational databases and backend data handling in real-world applications.

4.3.2 Performing CRUD Operations

I became proficient in implementing full CRUD (Create, Read, Update, Delete) operations using MySQL, Spring Boot, and Spring Data JPA. These operations were integrated into RESTful APIs developed for different modules of the café management system. For example, Create operations were performed using `save()` methods to add new users, cafés, menu items, bookings, and orders into the database. Read operations were implemented using methods like `findAll()`, `findById()`, and custom query methods to retrieve required data.

Update operations were handled using update operations and `save()` methods to modify existing records such as order status, bookings, and menu details. Delete operations were performed using `deleteById()` and `delete()` methods to remove records when necessary. I also implemented filtering and searching features using JPA query methods for café search, order tracking, booking history, and event management. This improved my understanding of backend data handling and database operations.

4.3.3 Real-World Use in Internship Projects

MySQL was used as the primary database system in several important modules developed during the Web-Based Coffee Ordering and Management Platform project. The database was responsible for storing customer details, café information, menu items, orders, bookings, payments, refunds, and event data. MySQL efficiently handled structured application data and maintained proper relationships between different system modules.

One of the important concepts I implemented was relational mapping using Spring Data JPA and Hibernate. Entity relationships helped connect users with orders, cafés with menu items, and bookings with payments through foreign key associations. I also implemented filtering, searching, and sorting operations for features such as café search, order tracking, booking history, customer management, and event reservations.

Another valuable learning experience was managing relational tables and optimized queries for handling large amounts of application data efficiently. Features such as indexing, validations, constraints, and entity mappings improved performance, maintained data consistency, and ensured secure backend operations throughout the application. This practical implementation enhanced my understanding of database design, query optimization, data integrity, and efficient data management in real-world full-stack web applications.

4.4 Frontend Development with React.js

Frontend development was an important part of my Web-Based Coffee Ordering and Management Platform project, where I gained practical experience building responsive and interactive user interfaces using React.js. React.js helped in developing reusable components, dynamic pages, and smooth user interactions for modules such as customer dashboard, café management, menu browsing, cart handling, bookings, payments, and event management. Through project development, I learned how to efficiently connect frontend components with backend APIs and manage application flow in a real-world environment.

React.js provided a flexible and component-based approach for developing modern web applications. I used React Hooks, routing, state management, API integration, conditional rendering, and form handling to build different features of the platform. During the project, I developed UI modules such as login and registration pages, dashboards, booking forms, order tracking pages, payment sections, and AI assistant interfaces using real-time data from Spring Boot backend APIs.

Additionally, React.js helped in improving the overall responsiveness and user experience of the café management platform by enabling faster rendering and seamless page navigation. The component-based architecture allowed efficient reuse of UI elements such as navigation bars, forms, cards, tables, and modals across multiple modules. This reduced repetitive code and made the frontend easier to maintain, update, and scale as new features were added to the application.

I also learned how to manage dynamic data and asynchronous operations in React applications. Features such as café search, live menu updates, booking availability, order tracking, and payment status were handled using API calls integrated with the Spring Boot backend. Loading indicators, conditional rendering, validation handling, and responsive layouts were implemented to ensure smooth interaction and better usability for customers, café owners, chefs, and waiters across different devices.

During the development process, I also gained experience in implementing secure authentication and role-based access control within the React application. Different dashboards and functionalities were displayed dynamically based on user roles such as Customer, Café Owner, Chef, Waiter, and Administrator.

4.4.1 Setting Up and Structuring a React.js Project

My first step in learning React.js was setting up projects using Create React App and Vite, which provided a structured environment for frontend development. I explored the project structure by working with folders such as components, pages, services, assets, and styles to maintain proper code organization. I also learned routing using React Router, where different pages and components were connected through defined routes, helping me understand navigation and application flow in React applications.

I also became comfortable working with important React features such as React Router for page navigation, reusable components for modular UI design, and React Hooks like `useState` and `useEffect` for state and lifecycle management. The component-based architecture and flexible routing system in React.js helped simplify frontend development, improve code reusability, and make the growing project easier to manage and maintain.

4.4.2 Building Pages and Components with React.js

In the frontend development of my project, I used React.js to build reusable, responsive, and user-friendly components for different modules of the café management platform. I created layouts and UI sections using component-based architecture, which helped maintain consistency and reduce repetitive code throughout the application.

I learned how to create reusable components for dashboards, menus, bookings, and forms. I also used React Router for navigation between different pages. Dynamic forms, modals, tables, and menu sections were developed for customer and café operations. User input and application state were managed using `useState` and `useEffect` hooks.

Component reusability and modular development helped improve maintainability, scalability, and overall frontend performance of the application. Additionally, React.js helped in creating interactive and responsive user interfaces for different user roles such as customers, café owners, chefs, and waiters. API integration using `Axios` and `Fetch` methods allowed real-time data updates for bookings, orders, payments, and menu management. This improved the overall user experience by enabling smooth navigation, faster page rendering, and dynamic content updates without reloading the entire application. The use of React.js components and modern frontend development practices enabled the platform to deliver a seamless, efficient, and visually appealing experience for all users while supporting future feature enhancements.

4.4.3 Data Fetching and Integration with Backend

In my project, React.js was integrated with the Spring Boot backend to fetch and manage dynamic data for different modules of the café management platform. I learned how to connect frontend components with backend REST APIs using Axios and Fetch API for smooth communication between the client and server.

I fetched data such as café details, menus, bookings, orders, payments, and customer information from backend APIs. JWT tokens were added in request headers for secure and role-based API access. I also used `useState` and `useEffect` for API data handling. I also learned how to perform client-side data fetching using `useEffect` with Axios and Fetch API in React.js. This helped me connect the frontend with Spring Boot APIs to retrieve café details, menus, bookings, orders, payments, and other dynamic data from the MySQL database. I also implemented JWT token handling and role-based access while making secure API requests from the frontend.

As part of my internship project, I developed interfaces for authenticated and unauthenticated users, implemented protected routes, and created forms for bookings, orders, payments, and profile management. I also used conditional rendering and loading components to improve user experience during asynchronous operations.

4.4.4 State Management and Dynamic UI Behavior

State management was an important part of frontend development, where I managed dynamic data and UI behavior using React.js hooks and techniques. `useState` and `useReducer` were used to manage local state like forms, cart, and orders. `useContext` was used to handle global data like user information and authentication status. Conditional rendering was implemented to display UI based on user roles such as customer, admin, chef, and waiter, along with order and booking status.

In booking, ordering, and profile modules, conditional rendering was used to show UI based on user selections like café choice, table availability, and event type. Form handling, file uploads, dropdowns, and validation were implemented using event handlers and `useEffect` for data updates and synchronization.

4.4.5 Real-World Application and Final Project Implementation

The practical outcome of my frontend development was the café management and customer service platform, where I applied all learned concepts in a real-world full-stack scenario. I built a responsive UI using React.js for customers, café owners, chefs, and waiters. Role-based routing and protected routes were implemented for secure

access control. The frontend was connected with the Spring Boot backend for handling orders, bookings, payments, and real-time updates.

Authentication and sessions were managed using JWT with secure login and profile-based access control. Dynamic data such as menus, orders, bookings, and user profiles were fetched and displayed using API integration.

This project helped me understand real-world system flow, including multi-role interaction, frontend-backend communication, and dynamic UI rendering in a full-stack café management system. It also strengthened my skills in API integration, state management, authentication handling, and building scalable user interfaces for complex business workflows.

Additionally, the frontend was designed using reusable React components and responsive layouts to provide a smooth user experience across desktop and mobile devices. Features such as dashboards, menu cards, booking forms, payment sections, notifications, and order tracking pages were developed with interactive UI elements and dynamic rendering. Conditional rendering techniques were also used to display different functionalities based on user roles such as Admin, Customer, Café Owner, Chef, and Waiter, ensuring secure and personalized access throughout the platform.

Another important achievement during the implementation phase was integrating real-time communication and efficient state management within the application. Live order updates, booking confirmations, and payment status notifications improved coordination between customers and café staff. Error handling, loading states, and validation mechanisms were also implemented to improve frontend reliability and user experience. This complete implementation provided practical understanding of building scalable enterprise-level applications while improving my confidence in handling real-world frontend development challenges and full-stack application workflows.

Furthermore, the successful implementation of the Coffee Ordering and Management Platform demonstrated how modern web technologies can be combined to automate café operations efficiently. The project provided valuable hands-on experience in developing, integrating, testing, and deploying a complete full-stack application while addressing real-world business requirements.

4.5 Version Control with Git and GitHub

Version control played an important role in managing and organizing the development of the café management project. During the development phase, I used Git and GitHub to track code changes, maintain project history, and ensure smooth development workflow. Git was used to manage different development branches for frontend features, backend APIs, bug fixes, and UI improvements without affecting the main project codebase.

GitHub provided a centralized platform for storing project repositories, tracking updates, resolving merge conflicts, and maintaining backup of the application source code. This version control workflow improved collaboration, code stability, and overall project management throughout the development process. It also improved code organization, simplified debugging, and helped safely manage project updates during development. Regular commits and branch management ensured a stable and efficient development.

Git was used for tracking changes in the codebase, creating branches for different features, and managing version history efficiently. GitHub was used as a remote repository to store the project code and support collaboration, code backup, and version synchronization. It also helped in managing pull requests and maintaining an organized workflow across different modules of the project.

This approach helped in maintaining a clean and structured codebase, improving development efficiency, and ensuring proper version control throughout the project lifecycle. It also made collaboration easier and reduced conflicts during integration of different features.

4.5.1 Understanding Git Basics and Local Version Control

At the beginning of my training, I was introduced to Git concepts such as repositories, commits, branches, and merges. I practiced setting up local repositories using basic commands like `git init`, `git add`, and `git commit -m "Initial commit"`. These fundamentals helped me understand how Git tracks changes in code and how each commit represents a snapshot of the project at a specific stage. I also learned to stage files selectively, commit changes with meaningful messages, and manage different development flows using branches.

Working with Git locally further helped me practice key operations such as creating and switching branches, merging features into the main branch, viewing

commit history, and resolving merge conflicts. This local version control allowed me to develop and test features independently without affecting the main codebase, ensuring stability and safer development. Additionally, I learned how to collaborate using remote repositories through platforms like GitHub. I practiced pushing local code to remote repositories, pulling updated changes from team members, and synchronizing branches during development.

This experience improved my understanding of collaborative software development workflows and helped me maintain proper version control while working on different modules of the café management platform. It also taught me the importance of code backup, project tracking, and maintaining clean commit histories in real-world development environments.

4.5.2 Collaborative Development Using GitHub

GitHub became the central platform for pushing code from my local environment and collaborating with teammates. GitHub was used as the central platform for managing and collaborating on the café management project. The entire codebase was maintained in a remote repository, enabling smooth development, version tracking, and teamwork across different modules.

I followed the typical GitHub collaboration workflow. First, feature branches were created where new features like ordering, booking, and admin panels were developed separately to keep the main branch stable. All updates were regularly pushed to the remote repository using `git push origin feature-branch`. After completing each feature, Pull Requests (PRs) were created with proper descriptions for review and integration. Code reviews and suggestions were performed to improve structure, readability, and ensure proper implementation of features. After approval, PRs were merged into the main branch, and merge conflicts were resolved whenever necessary.

GitHub helped in maintaining proper version control, improving collaboration, tracking changes efficiently, and ensuring a well-organized and consistent project workflow throughout development. It also made team coordination easier by providing clear visibility of all updates and contributions. GitHub provided a secure and reliable environment for project backup, documentation, and progress monitoring. Features such as commit history, issue tracking, and repository management helped maintain development transparency and ensured smooth coordination among team members throughout the project lifecycle.

4.5.3 Best Practices and Workflow Improvements

Throughout the development of the café management project, I followed industry-standard Git and GitHub best practices to maintain a clean and efficient workflow. Meaningful commit messages were written for better project tracking and readability. Feature branches were used for implementing modules like ordering, booking, and admin functionalities. The .gitignore file was used to exclude unnecessary files such as node modules and build files. Code reviews were performed before merging to ensure error-free and optimized code. I also practiced cloning repositories, forking projects, and contributing through pull requests, which improved my understanding of collaborative development and real-world Git workflows.

Additionally, branch management strategies were carefully followed to maintain stability during development. Separate branches were created for frontend development, backend APIs, bug fixes, and testing purposes. This helped isolate changes and reduced the risk of affecting the main application codebase during feature implementation. Before merging branches, proper testing and validation were performed to ensure compatibility and smooth integration between different modules of the café management platform.

Another important practice followed during development was maintaining regular backups and synchronization using GitHub repositories. Frequent commits and pushes helped track project progress efficiently and prevented loss of important code changes. GitHub Issues and Pull Request discussions also improved communication and task management during collaborative development. This workflow enhanced project organization, simplified debugging, and supported efficient teamwork throughout the development process. Furthermore, version control practices helped in monitoring code history and identifying changes introduced in specific features or updates.

4.5.4 Learning from Real-World Scenarios

During the development of the café management project, Git and GitHub played an important role in handling real-world development challenges and improving workflow efficiency. I collaborated by pushing code to shared repositories across different modules. Merge conflicts were resolved when multiple changes were made to the same files such as UI, API integration, and booking logic. Commits were used to track project progress and identify changes in different features. Previous versions of the project were restored whenever bugs occurred.

In one instance, a recent update caused a UI issue in the booking module, and Git history helped identify the problematic commit quickly. This experience improved my understanding of Git as not only a version control tool but also a powerful debugging and recovery system. It also helped me fix issues faster and maintain project stability during development.

Additionally, working on real-world development scenarios taught me the importance of maintaining proper coordination and communication while handling collaborative projects. Since multiple modules such as frontend UI, backend APIs, payment integration, and booking management were being developed simultaneously, GitHub helped manage updates systematically and reduced confusion between different code changes. This experience improved my ability to work in a team-based development environment and understand professional software development workflows.

Another valuable learning experience was handling branch synchronization and deployment-related challenges. Before merging new features into the main branch, careful testing and code validation were performed to avoid breaking existing functionalities. In situations where newly added features introduced unexpected bugs or performance issues, Git branches and commit history helped isolate the problem quickly and restore stable versions of the application. This process improved debugging efficiency and minimized downtime during development.

Furthermore, these real-world experiences highlighted the importance of maintaining clean code, proper documentation, and effective version control practices throughout the development lifecycle. By working with Git and GitHub in a collaborative environment, I gained practical knowledge of software maintenance, team coordination, release management, and continuous improvement processes that are commonly followed in professional software development organizations.

Moreover, working with GitHub exposed me to industry-standard development practices such as branch management, code reviews, issue tracking, and collaborative problem-solving. These experiences enhanced my ability to contribute effectively to team projects, manage development timelines, and ensure software quality through systematic testing and version control. The knowledge gained from these activities will be valuable for future software engineering and full-stack development projects.

Chapter 5

INTERNSHIP PROJECT CONTRIBUTION

After completing the foundational training phase of the Infosys Springboard Virtual Internship 6.0, I moved into the project implementation stage where I transitioned from learning concepts to applying them in real-world development work. This phase marked the beginning of my hands-on contribution to a structured, production-oriented application. My initial focus was to understand the complete product in depth—its purpose, core workflows, user roles, and the technologies used to build it. I analyzed how the frontend communicates with the backend, the structure of APIs, the folder architecture, and how each feature contributes to the overall system. This helped me align my development approach with real-world application requirements and user expectations.

The project was built using modern full-stack technologies including React.js (and Next.js where applicable) for the frontend, Node.js and Express.js for backend services, and MongoDB for database management. The UI was enhanced using Chakra UI for better design consistency, and Socket.io was used for real-time communication features. I explored the entire codebase by inspecting components, services, and API layers to understand coding patterns and system design. I also studied the different user roles such as admin, client, and other role-based users, each having specific permissions and access control logic, which is essential for secure and scalable application development.

Once I gained a clear understanding of the system, I started working on my initial technical tasks, which included identifying and resolving existing errors and warnings in the application. This phase involved debugging both frontend and backend issues. On the frontend, I worked on fixing JSX errors, UI inconsistencies, and Chakra UI-related warnings, while on the backend, I resolved API response issues, controller logic errors, and data handling mismatches. This experience helped me understand how even small coding mistakes can affect the stability and performance of a live application and how important debugging is in real-world development environments.

After completing debugging tasks, I progressed to more complex contributions involving feature enhancements and code refactoring across both frontend and backend. I worked on improving code structure for better scalability, creating reusable components, and ensuring smooth data flow between different modules. I also

contributed to implementing new features such as dynamic dashboards and real-time functionalities, while maintaining clean coding practices. Overall, this phase of the internship helped me gain strong practical exposure to full-stack development, system understanding, debugging, and real-world software engineering practices.

5.1 Initial Product Understanding and System Analysis

The application was structured around multiple user roles—primarily Admin, Café Owner, Chef, Waiter, Event Staff, and Customer—each with different levels of access and responsibilities. I mapped out each role’s functionalities along with route protections and frontend component rendering based on authenticated user roles using JWT and Spring Security. This helped in understanding how the system dynamically adapts dashboards, bookings, orders, and payment modules according to the logged-in user. I also analyzed how all system data such as customer profiles, café details, bookings, and payments were stored and managed using MySQL with secured REST APIs. It shows the home page interface of the CaféConnect platform with navigation, login, registration, and café browsing options.

From a technical perspective, the platform follows a modular full-stack architecture using React.js for the frontend and Spring Boot for the backend. The frontend is built with reusable components and responsive design principles, while the backend follows a layered structure with controllers, services, and repositories. I also reviewed API communication flow, state management, and error handling between frontend and backend systems. Additionally, real-time features like order tracking and booking updates were studied for future WebSocket integration. Fig 5.2 shows the customer café browsing dashboard where users can search cafés, view listings, and access café details through an interactive interface.

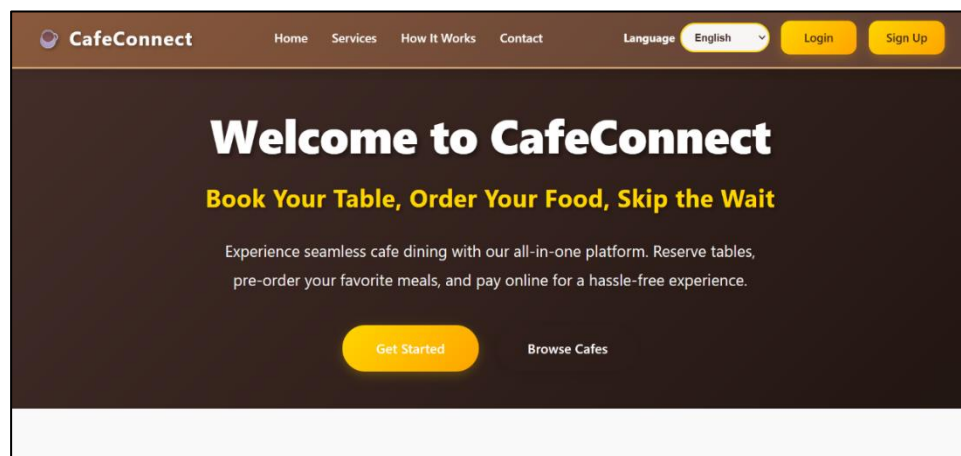


Fig 5.1: Application Screenshot Look Alike

Fig 5.1 shows the Café Connect home page interface, designed as a simple and interactive entry point to the platform. It includes navigation options like Home, Services, How It Works, and Contact, along with language selection support. The page also provides login and registration options, and quick access buttons such as “Get Started” and “Browse Cafes” for easy café exploration. The interface is developed using React.js with responsive design for smooth user experience across devices.

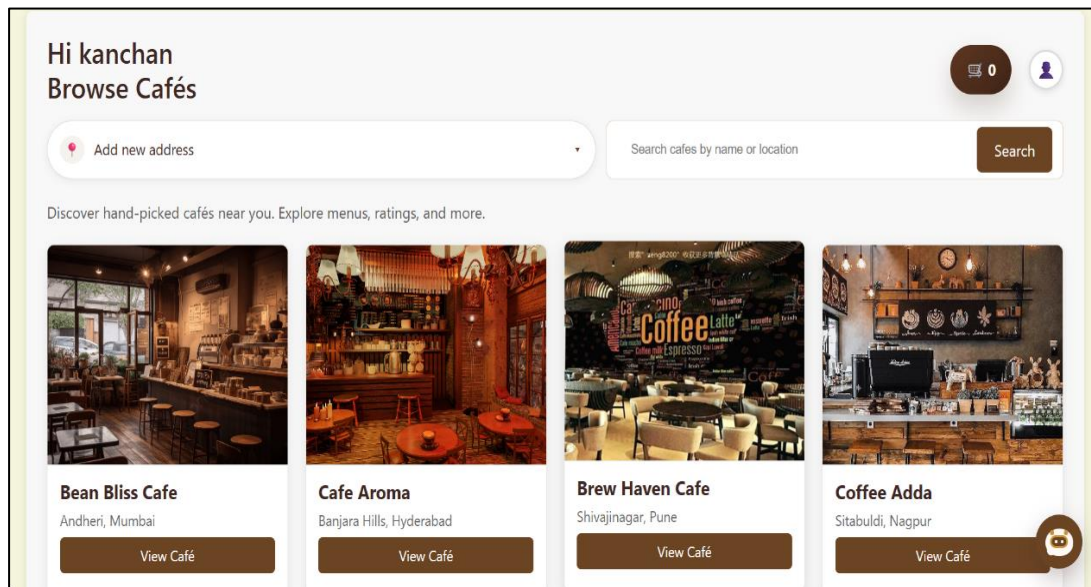


Fig 5.2: Coffee Ordering Services

Fig 5.2 shows the customer café browsing dashboard of the CaféConnect platform, where users can search and explore cafés based on location and preferences. It includes features like café search, address selection, cart access, and profile management. Each café is displayed in card format with image, name, and location, along with an option to view details. The interface uses a component-based React.js design with dynamic data rendering from backend APIs to enhance user experience and simplify café browsing.

5.2 Frontend Console Cleanup and API Error Resolution

One of my key responsibilities during the development of the Web-Based Coffee Ordering and Management Platform was resolving frontend console warnings and API-related errors in the application. These issues affected the performance, maintainability, and smooth functioning of the system. My primary focus was on fixing UI-related warnings generated from React and Chakra UI components used in café booking, menu management, and ordering modules. I reviewed the frontend components carefully and

corrected styling issues, invalid prop usage, and inconsistent component structures to improve the overall frontend stability.

I also encountered several React warnings such as missing key props in dynamic lists and improper nesting of components, which could lead to rendering problems and unstable UI behavior. These issues were resolved by restructuring component hierarchies and ensuring proper key assignment while rendering café menus, bookings, and order details. This improved the responsiveness, reliability, and performance of the frontend application.

In addition to frontend cleanup, I worked on debugging API-related errors occurring during communication between the React frontend and Spring Boot backend. Many issues were caused by invalid API responses, missing authentication headers, token validation failures, and incorrect request handling. I verified API integrations related to login, food ordering, table booking, payment processing, and order tracking while implementing proper error-handling mechanisms such as try-catch blocks and user-friendly error messages. These improvements made the system more stable and enhanced the overall user experience.

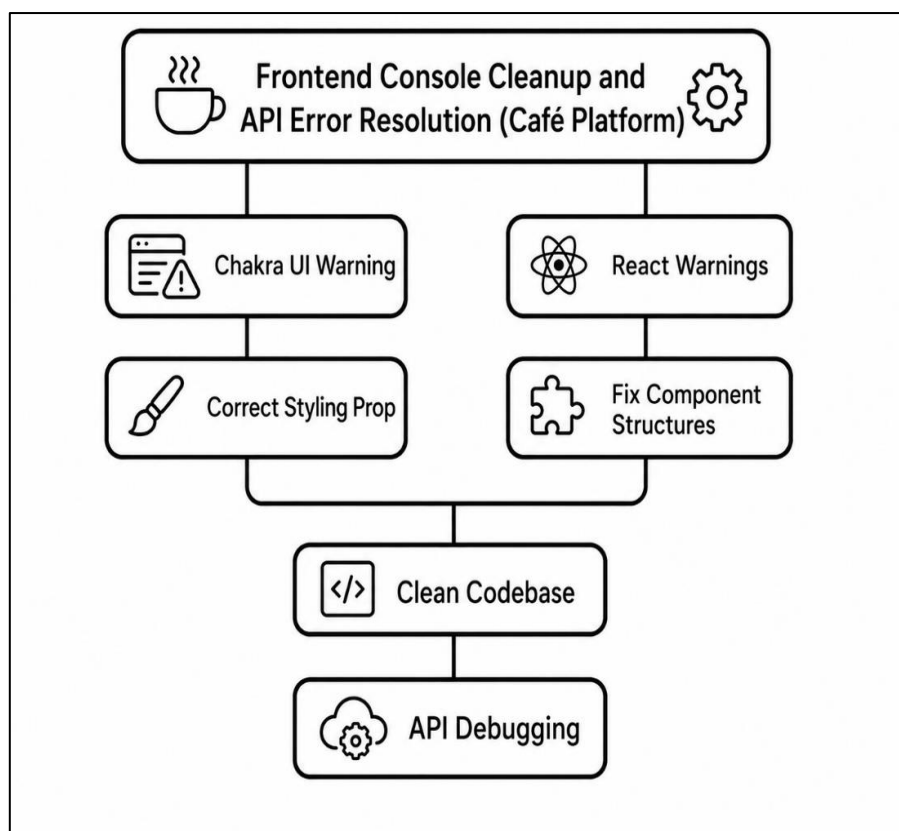


Fig 5.3: Errors and Warning Resolution

Fig 5.3 shows the workflow followed during the frontend console cleanup and API error resolution process implemented in the café management platform. Overall, this phase significantly improved the cleanliness, maintainability, and operational stability of the system. The removal of console warnings enhanced frontend code quality, while API debugging strengthened the communication between the React frontend and Spring Boot backend services. This task also helped me gain practical experience in real-time debugging, frontend optimization, secure API integration, and production-level issue handling. It trained me to proactively identify hidden issues that could otherwise become major problems during deployment or live system usage.

5.3 Backend Code Optimization Through Separation of Concerns

As part of improving the performance and maintainability of the Web-Based Coffee Ordering and Management Platform, I optimized the backend structure by applying the Separation of Concerns (SoC) principle. The backend architecture was reorganized into separate controller, service, and repository layers to improve code readability, scalability, and maintainability.

The first step of this optimization process involved refactoring route files related to café management, food ordering, table booking, payments, and user authentication. I created dedicated controller functions responsible for handling HTTP requests, validating input data, and returning structured API responses with proper status codes. This helped separate request-response handling from the core business logic and made the backend workflow more organized and efficient.

Next, I developed service modules to manage reusable business logic and database operations within the Spring Boot backend. Functionalities such as customer registration, order processing, menu management, booking validation, payment verification, and order tracking were shifted into dedicated service classes. This approach reduced code duplication, improved modularity, and made the application easier to maintain and test. It also simplified the addition of future features such as caching, logging, and validation without affecting existing business logic.

To verify the reliability and performance of the optimized backend system, I tested all APIs using Postman and validated response formats, authentication flows, status codes, and execution speed. I carefully monitored APIs related to customer operations, café owner management, chef order handling, and waiter services to ensure smooth communication between frontend and backend systems.

Additional exception handling and validation mechanisms were implemented to improve application stability and secure API execution. Fig 5.4 illustrates the workflow followed during the backend code optimization process implemented in the café management platform. Overall, this optimization significantly improved backend maintainability, scalability, and performance while enhancing the quality of frontend-backend communication. This task also strengthened my understanding of clean architecture, modular backend development, RESTful API design, and real-world full-stack application development practices.

Additionally, it helped me gain practical experience in organizing large-scale backend systems and improving code reusability for future enhancements. The optimization process also reduced debugging complexity and made the backend more efficient, secure, and easier to maintain during production deployment.

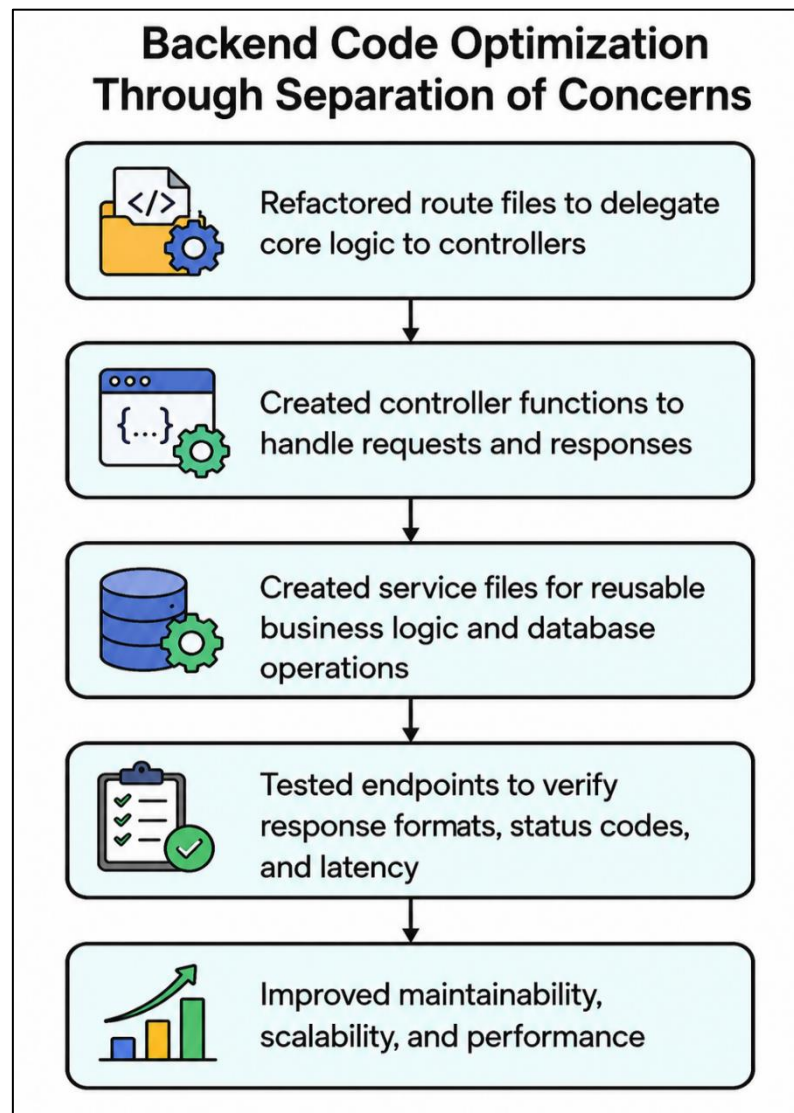


Fig 5.4: Backend Code Optimization

5.4 Middleware Implementation for Role-Based Access Control

One of the important backend responsibilities I handled during the development of the Web-Based Coffee Ordering and Management Platform was implementing middleware for authentication and role-based access control (RBAC). Middleware in the Spring Boot backend acted as a security layer that processed requests before they reached the API endpoints. Its primary purpose was to ensure that only authenticated users could access protected resources and that different user roles such as Admin, Café Owner, Chef, Waiter, Event Staff, and Customer had controlled access to specific functionalities within the platform.

The first step in this implementation process was developing authentication middleware using JWT (JSON Web Token). The middleware extracted the token from incoming request headers, validated it using secure secret keys, and decoded the token payload to retrieve user identity and role information. If the token was expired, invalid, or missing, the system returned an unauthorized access response. This JWT-based authentication mechanism secured APIs related to login, café management, food ordering, table booking, event reservations, payment processing, and order tracking within the platform.

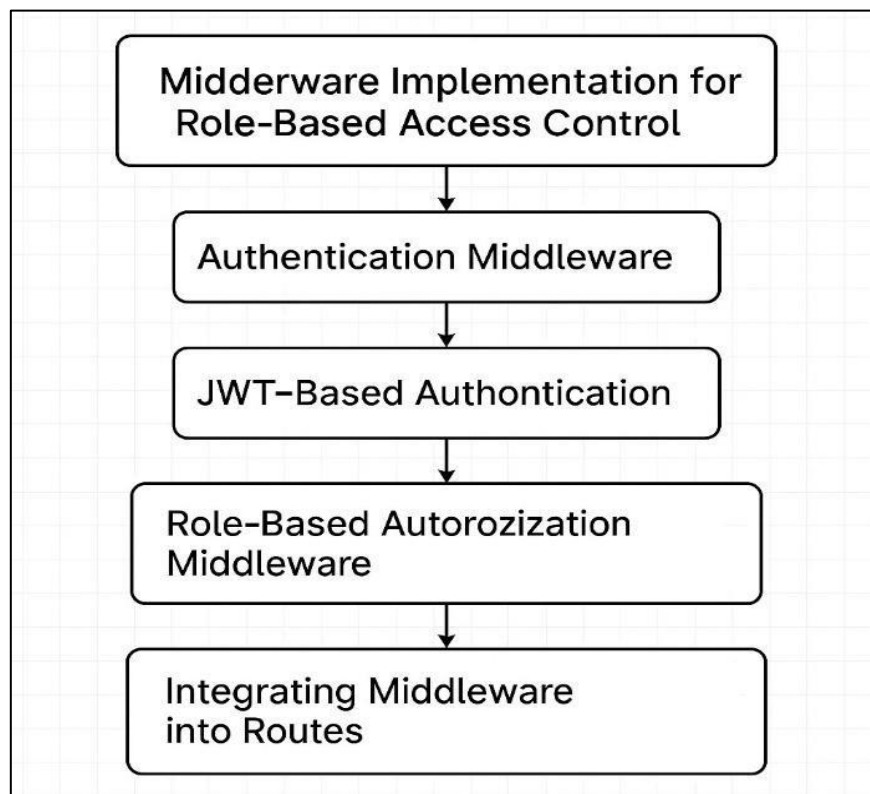


Fig 5.5: Middleware Implementation

Fig 5.5 shows the workflow followed during the middleware implementation process for role-based access control in the café management platform. Through this implementation, I gained practical experience in securing REST APIs using JWT authentication, validating user sessions, and implementing scalable authorization mechanisms. The role-specific access restrictions also created a strong foundation for future enhancements such as audit logging, activity tracking, advanced permission management, and secure feature-based access control. This task improved my understanding of secure backend architecture and the importance of middleware in building maintainable full-stack applications.

After successfully implementing authentication, I extended the system by developing role-based authorization middleware to restrict access according to user responsibilities within the application. The middleware verified whether an authenticated user possessed the required role to perform a specific operation. For example, only Admin users could manage café owner accounts, Café Owners could manage menus and staff, Chefs could update food preparation status, and Waiters could manage served orders. The middleware was designed in a reusable and dynamic manner so that the same logic could be applied across multiple backend modules without code duplication.

Integrating these middleware components into the backend route structure required securing important APIs and modifying request workflows throughout the application. I ensured that modules related to bookings, payments, café management, and staff operations were protected using authentication and authorization middleware. This implementation improved the security, modularity, and maintainability of the backend system while preventing unauthorized access to sensitive operations within the café management platform.

5.5 Implementation of Role-Specific Functionality and Data Flow

In the Web-Based Café Ordering and Management Platform, multiple users such as Admins, Café Owners, Customers, Chefs, and Waiters interact within a single system. Implementing role-specific functionality was important to provide personalized dashboards, controlled access, and secure operations for each user. One of the major tasks in this project was developing a role-based system that dynamically managed features and permissions according to the authenticated user's role.

The implementation started on the backend using Spring Boot, where controller functions and REST APIs were created for role-specific operations such as café management, table booking, menu handling, order tracking, payment management, and staff operations. These APIs were secured using Spring Security and JWT authentication to ensure that only authorized users could access specific modules and functionalities. Separate APIs were developed for different user roles to maintain secure access control and smooth system operations. Validation and authorization checks were also implemented to protect sensitive data and improve system security.

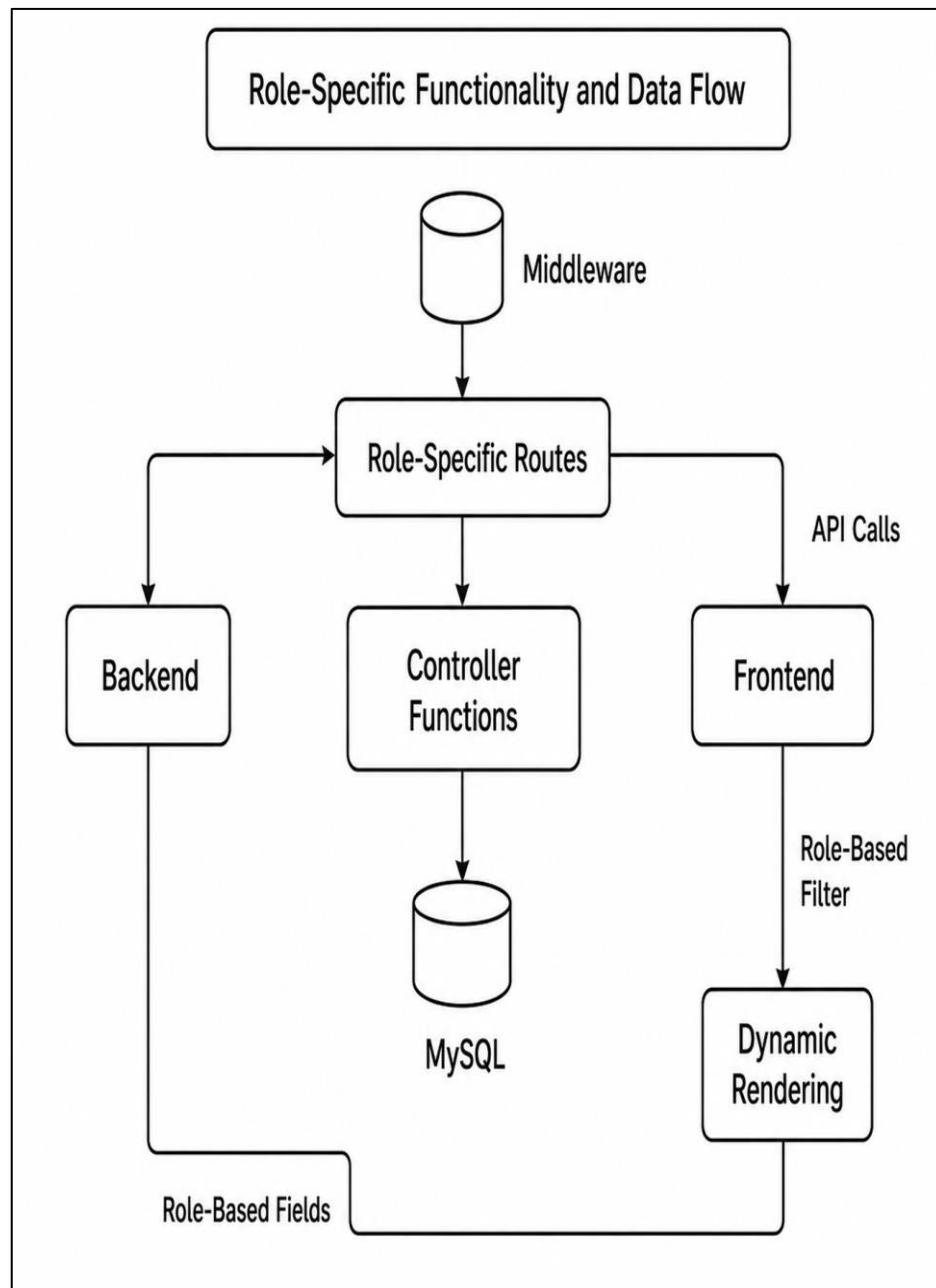


Fig 5.6: RBAC Implementation

Fig 5.6 shows the flow used to implement Role-Based Access Control (RBAC) in the system. On the frontend, React was used to create dynamic dashboards and responsive user interfaces. Backend APIs were consumed using Axios and Fetch API, and components were rendered conditionally based on the logged-in user's role. Features such as booking management, menu updates, payment sections, and order tracking were displayed only to authorized users.

After completing the API layer, the MySQL database schema was designed to support role-based relationships and workflows. Tables such as Users, Roles, Orders, Cafes, Bookings, Payments, and Menus were connected using relational mappings. Additional role-specific fields were included to manage permissions, order statuses, and booking information efficiently. Real-time order updates were also implemented to improve coordination between chefs and waiters.

This implementation improved understanding of full-stack development by combining backend APIs, database management, authentication, and frontend rendering with role-based access control. It also enhanced knowledge of secure system design, real-time workflow handling, and personalized user experiences in enterprise-level web applications.

5.6 Real-Time Chat Functionality Using Socket.IO

One of the most important features implemented in the Web-Based Café Ordering and Management Platform was the real-time order tracking and notification system. This functionality enabled instant communication between customers, chefs, waiters, and café owners during the order process. To implement this feature, WebSocket technology was used to establish continuous two-way communication between the frontend and backend, allowing live updates without refreshing the page.

The first step was integrating WebSocket support into the Spring Boot backend and configuring it to handle real-time client connections. On the frontend side, React components were connected to the WebSocket server to listen for live updates. Whenever a customer placed an order, the request was immediately sent to the backend and displayed on the chef and waiter dashboards in real time. This persistent connection allowed smooth communication between different modules of the system.

Additionally, real-time notifications were also implemented for table bookings, payment confirmations, and event reservations. Customers received instant updates whenever their booking was approved, payment was completed, or order status changed

within the system. This helped improve transparency and allowed users to track their activities without manually refreshing the application.

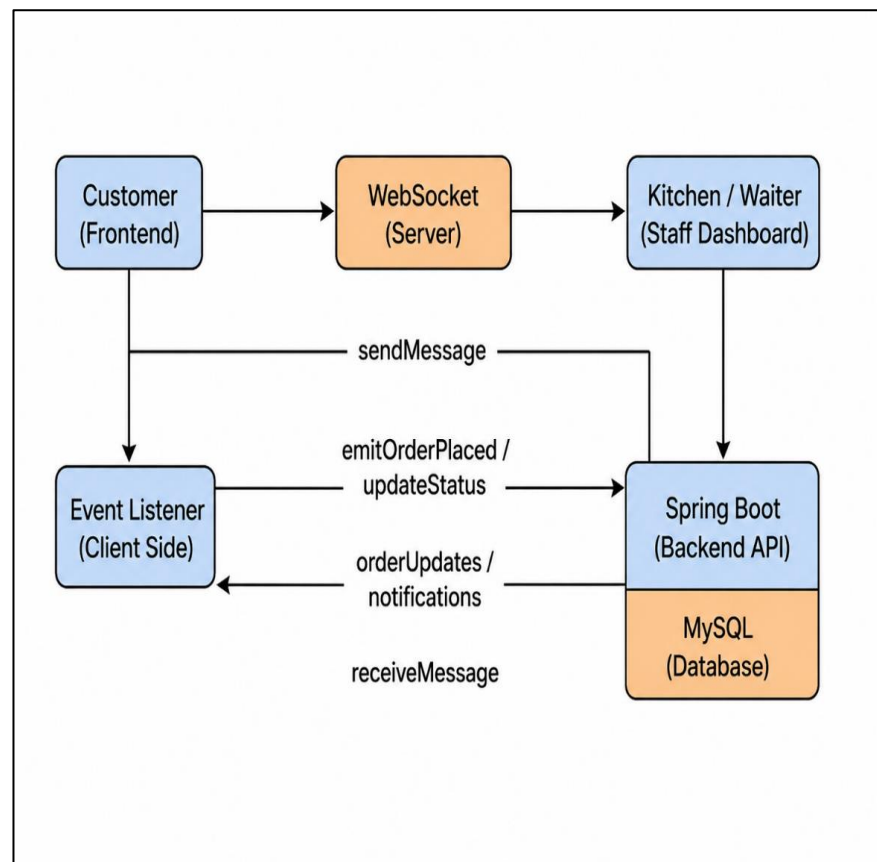


Fig 5.7 shows the flow used to implement the real-time order tracking system in the café management platform. For backend data handling, MySQL was used to store order details, customer information, payment records, and order statuses. Additional APIs were also developed to fetch previous orders, booking details, and payment history whenever required by customers or café owners. Each order record was connected with customer profiles, table reservations, timestamps, and payment information to maintain proper tracking and data consistency within the system. This helped café owners and staff monitor ongoing operations efficiently while providing customers with accurate real-time updates about their orders and bookings.

Additionally, the real-time tracking system improved coordination between customers, chefs, and waiters by ensuring smooth communication of order progress within the café environment. Notifications and status updates were displayed dynamically on the frontend whenever an order moved from “Order Placed” to “Preparing,” “Ready,” and finally “Served.” The implementation also reduced manual confusion in order handling, minimized waiting time for customers, and improved overall service efficiency. Security and reliability were important aspects of this feature. Only authenticated users were allowed to establish WebSocket connections using JWT-based authentication and token validation. This prevented unauthorized users from accessing live order information within the system. Error handling mechanisms were also implemented to manage disconnection, reconnection attempts, and communication failures efficiently.

Building this module improved understanding of event-driven architecture, real-time state synchronization, and WebSocket management in full-stack applications. It also demonstrated the difference between traditional REST APIs and real-time WebSocket communication in scenarios requiring instant updates. Overall, the real-time notification system strengthened full-stack development skills and improved the ability to build scalable and interactive web applications.

5.7 Report Functionality with Role-Based View Access

The development of the Web-Based Café Ordering and Management Platform, one of the important modules implemented was the report functionality with role-based view access. This module was designed to manage and display reports related to café bookings, customer orders, payments, reservations, and staff activities. The feature required both backend and frontend development along with secure access control to ensure that different users could access only the information relevant to their roles.

The main requirement of this module was to allow café owners and administrators to generate and manage multiple report entries efficiently. To support this functionality, the MySQL database schema was designed to handle report-related data such as orders, payments, reservations, customer details, and event bookings. On the backend, REST APIs were developed using Spring Boot to process report data, validate inputs, and securely store and retrieve information from the database.

After implementing the backend APIs, the frontend functionality was developed using React. Retrieved report data was passed dynamically across components using

props and state management. Reusable UI components were created to display booking reports, payment summaries, customer order history, and staff activity reports. Conditional rendering techniques were used to dynamically display reports based on available data and user roles, ensuring responsive and scalable frontend behavior. In parallel, role-based access control was implemented for the report module using Spring Security and JWT authentication. Different access permissions were assigned to Admins, Café Owners, Customers, Chefs, and Waiters according to their responsibilities within the system. For example, administrators and café owners could manage and export reports, while customers could only view their own order and payment history. Unauthorized users were restricted from accessing sensitive report data through backend authorization and frontend role validation.

This implementation improved understanding of dynamic data handling, API integration, frontend rendering, and secure access management in full-stack applications. It also provided practical experience in implementing role-based permissions, reusable frontend components, and database-driven reporting systems. Overall, the report functionality demonstrated the ability to build scalable, secure, and user-friendly enterprise modules within a multi-user café management platform.

5.8 UI Design and Responsive Frontend Development

A visually consistent and user-friendly interface is an important part of any modern web application. During the development of the Web-Based Café Ordering and Management Platform, significant work was done to improve the frontend user experience by enhancing layouts, styling components, improving responsiveness, and making the interface easier to use for customers, café owners, chefs, waiters, and administrators. The main focus was on building clean, reusable, and maintainable UI components while ensuring the application appeared professional across different screen sizes. The frontend design was enhanced using responsive layouts, reusable UI components, and interactive elements to improve user experience across different devices. Consistent styling, navigation flow, and component structure helped make the platform more professional, user-friendly, and visually organized.

Additionally, special attention was given to improving accessibility and usability within the platform by simplifying user interactions and reducing unnecessary complexity in the interface. Features such as responsive dashboards, organized menu

displays, smooth navigation, and clear status indicators helped users perform operations more efficiently.

React and responsive styling techniques were used to create a structured and interactive user interface. Layout components and responsive design patterns were implemented to organize pages such as the café menu, cart, table booking, event booking, customer dashboard, payment section, and order tracking modules. Consistent spacing, typography, colors, cards, buttons, and form elements were used throughout the application to improve the visual appearance and overall usability of the platform.

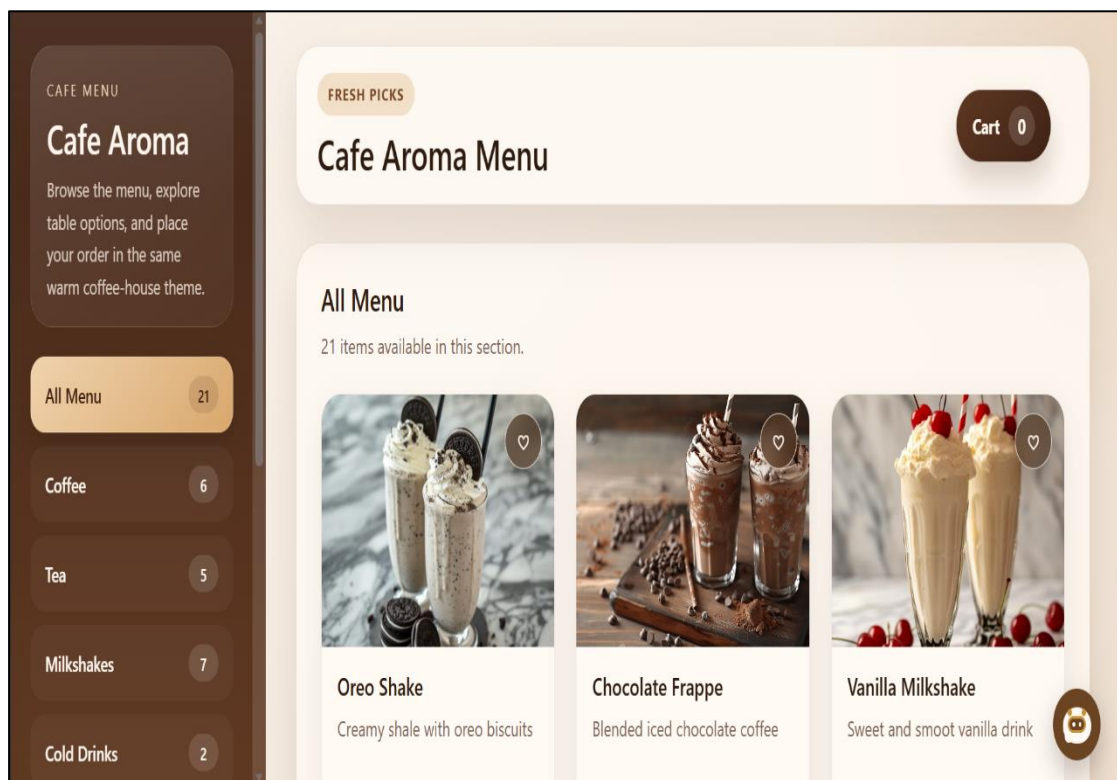


Fig 5.8: Screenshot of Look-alike application Implementation

Fig 5.8 shows the Coffee Platform user interface implementation, highlighting the frontend styling and layout improvements made during the project. The screenshot represents a clean and organized page where users can easily view menu items, booking options, cart actions, and other coffee shop services. It demonstrates the use of consistent colors, spacing, buttons, cards, filters, and responsive design to improve usability across devices. Overall, this figure reflects how UI enhancements helped make the Coffee Platform more professional, user-friendly, and visually consistent.

Interactive UI features such as conditional buttons, status badges, modals, notifications, and responsive navigation sections were also implemented to improve usability. Customers could easily browse menu items, place orders, book tables, reserve

events, and track orders through a simple and organized interface. Similarly, café owners and staff members were provided with clean role-based dashboards for managing menus, bookings, payments, and customer requests efficiently.

Another important aspect of this work was improving responsiveness and frontend consistency across devices. Layout issues such as overlapping content, inconsistent spacing, and alignment problems were fixed using flexible responsive design techniques. Reusable components and shared styling patterns were implemented to maintain consistency across forms, cards, dashboards, buttons, and tables. These improvements enhanced the maintainability of the frontend code and made the platform more professional, scalable, and user-friendly.

5.9 Code Optimization and Version Control Across Branches

As the development of the Web-Based Café Ordering and Management Platform progressed, maintaining an optimized and stable codebase became an important part of the project. One of the key tasks during this phase involved managing source code efficiently across different Git branches while implementing backend and frontend optimizations. This process was necessary because multiple modules such as bookings, payments, orders, dashboards, and real-time notifications were being developed simultaneously.

The primary objective was to ensure that optimized backend and frontend code changes were safely transferred from development branches to staging or main branches without affecting existing functionalities. Backend improvements included restructuring Spring Boot APIs into separate controller, service, and repository layers to improve code readability, scalability, and maintainability. On the frontend side, reusable React components and shared styling structures were implemented to reduce code duplication and improve performance across different modules.

Additionally, Git version control was used extensively to manage feature development, testing, debugging, and code integration across multiple branches. Separate branches were maintained for new features, bug fixes, and UI improvements to ensure safer development and easier collaboration. Before merging code into the main branch, proper testing and conflict resolution were performed to maintain application stability and avoid breaking existing functionalities.

5.10 Bug Fixing and Feature Enhancements

During the later stages of developing the Web-Based Café Ordering and Management Platform, major focus was given to improving the stability, performance, and usability of the application by resolving bugs and enhancing existing features. These improvements were important to ensure smooth system operation for customers, café owners, chefs, waiters, and administrators. Although several fixes were minor, they significantly improved the overall user experience and workflow efficiency within the platform.

One of the major responsibilities was monitoring the application during testing and real-time usage to identify and resolve issues quickly. This involved debugging both frontend and backend modules using browser developer tools, console logs, API response tracking, and network monitoring tools. On the backend side, Spring Boot controller logic, service methods, and MySQL queries were reviewed regularly to ensure smooth data processing and efficient communication between different system modules.

Feature enhancements included improving functionalities such as menu browsing, cart management, table booking, event reservation, payment handling, and order tracking. Additional improvements were also made in role-based dashboards for café owners, chefs, and waiters to provide clearer order visibility, better navigation, and improved workflow management. Responsive design fixes and reusable frontend components were implemented to maintain consistency across desktop and mobile devices.

This phase also improved understanding of clean coding practices, modular development, and maintainable application design. Since many fixes affected shared components and common workflows, careful testing and validation were performed before deploying updates. Git version control practices such as feature branching, pull requests, and isolated updates were followed to ensure code stability and avoid conflicts during collaborative development. Additionally, continuous testing and feature refinement helped improve the reliability and overall performance of the platform. User feedback and testing observations were considered while implementing enhancements, ensuring that the application remained user-friendly, efficient, and responsive.

CONCLUSION

The internship experience provided valuable practical exposure to full-stack web application development and significantly improved technical and problem-solving skills. Working on the Web-Based Café Ordering and Management Platform helped in understanding how frontend, backend, database management, authentication, and real-time communication work together in a real-world enterprise application. The project involved multiple modules such as customer management, café operations, table booking, event reservation, online payments, order tracking, and role-based dashboards, which provided hands-on experience in building scalable and user-focused systems.

Throughout the development process, work was carried out on both frontend and backend functionalities, including responsive UI development, API integration, role-based access control, database operations, real-time notifications, and workflow management. Technologies such as React, Spring Boot, MySQL, JWT authentication, WebSocket communication, and Git version control were actively used to implement and manage different system modules. These tasks helped improve debugging abilities, coding practices, API handling, and understanding of maintainable application architecture.

Overall, this project strengthened knowledge of full-stack development and provided a clear understanding of real-world software engineering practices. The experience helped bridge the gap between academic concepts and industry-level application development while improving confidence in developing secure, scalable, and user-friendly web applications. It also motivated continuous learning and further skill enhancement for future opportunities in the software development industry.

REFERENCES

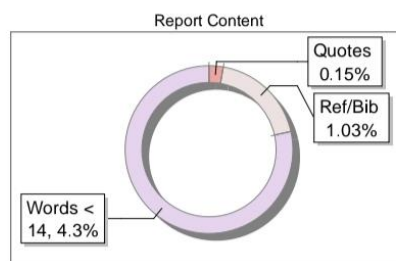
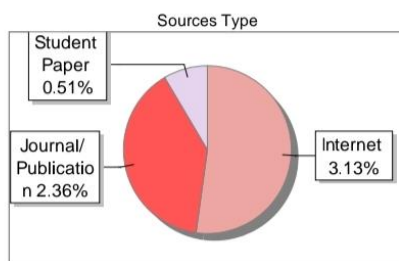
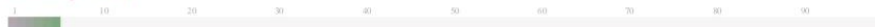
- [1] D. Flanagan, JavaScript: The Definitive Guide, 7th ed., O'Reilly Media, 2020.
- [2] C. S. Horstmann, Core Java Volume I – Fundamentals, 12th ed., Pearson Education, 2021.
- [3] Spring Boot Documentation, Spring Boot Reference Guide. [Online]. Available: Spring Boot Documentation
- [4] Spring Security Documentation, Spring Security Reference. [Online]. Available: Spring Security Documentation
- [5] React Documentation, React – A JavaScript Library for Building User Interfaces. [Online]. Available: React Documentation
- [6] R. Johnson, J. Hoeller, A. Arendsen, T. Risberg, and C. Sampaleanu, Professional Java Development with the Spring Framework, Wiley Publishing, 2005.
- [7] MySQL Documentation, MySQL 8.0 Reference Manual. [Online]. Available: MySQL Documentation
- [8] JWT.io, JSON Web Tokens Introduction. [Online]. Available: JWT Documentation
- Apache Maven Project, “Maven Documentation.” [Online]. Available: <https://maven.apache.org/guides>
- [9] Bootstrap Documentation, “Bootstrap Frontend Framework.” [Online]. Available: <https://getbootstrap.com/docs>
- [10] Stripe Documentation, “Stripe API Reference.” [Online]. Available: <https://stripe.com/docs/api>
- [11] Razorpay Documentation, Razorpay Payment Gateway API Documentation. [Online]. Available: Razorpay Documentation
- [12] M. Bauer and G. King, “Java Persistence with Hibernate,” IEEE Software, vol. 25, no. 6, pp. 90–92, Nov.-Dec. 2008.

Submission Information

Author Name	Dhutade Kanchan Raju
Title	New updated report coffee
Paper/Submission ID	5736536
Submitted by	rajurkar_am@mgmcen.ac.in
Submission Date	2026-05-27
Total Pages, Total Words	60, 16340
Document type	Project Report

Result Information

Similarity **6 %**



Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	0 %
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File

